Pricing Asian Option by the FFT with Higher-Order Error Convergence Rate under Lévy Processes

Chun-Yuan Chiu* Tian-Shyr Dai† Yuh-Dauh Lyuu‡

November 23, 2014

Abstract

Pricing Asian options is a long-standing hard problem; there is no analytical formula for the probability density of its payoff even when the process of the underlying asset follows the simple lognormal diffusion process. It is known that the density function of a discretely-sampled Asian option's payoff can be efficiently approximated by the Fast Fourier Transform (FFT). As a result, we can accurately price the option under more general Lévy processes. This paper shows that the pricing error of this approach, called the FFT approach, can be decomposed into the truncation error, the integration error, and the interpolation error. We prove that previous algorithms that follow the FFT approach converge quadratically. To improve the error convergence rate, our proposed algorithms reduce the integration error by the higher-order Newton-Cotes formulas and new integration rules derived from the Lagrange interpolating polynomial. The interpolation error is reduced by the higher-order Newton divided-difference interpolation formula. Consequently, our algorithms can be sped up by the FFT to achieve the same time complexity as previous algorithms, but with a faster error convergence rate. Numerical results are given to verify the efficiency and the fast convergence of our algorithms.

Keywords: pricing, Fast Fourier Transform, Asian option, Newton-Cotes integration formula

1 Introduction

This paper proposes a pricing scheme for discretely-sampled Asian options. The payoff of an Asian option depends on the average price of the underlying asset from the option's starting date to the maturity date. Its price is less subject to manipulation [15]. It is also useful for hedging transactions whose cost depends on the average price of the underlying asset (such as commodities like crude oil). A general review of Asian options in commodity markets can be found in [27]. However, it is hard to price Asian options because the probability distribution of the average price does not have a simple analytical expression [33]. (From now on, the underlying asset is assumed to be stock for convenience.)

Much past literature assumes that the stock price follows a lognormal diffusion process and approximates the density of the average stock price by (1) the geometric average of the stock price [25], (2) the lognormal density function [26], (3) the Edgeworth series expansion [31], (4) the Taylor expansion [37], or (5) the reciprocal Gamma distribution [29]. Many resulting closed-form approximation formulae may fail under extreme scenarios [17]. To improve the pricing accuracy, numerous

*Institute of Information Management, National Chiao-Tung University, 1001 Ta Hsueh Road, Hsinchu 300, Taiwan, ROC. E-mail: r94922072@ntu.edu.tw.

†Department of Information and Finance Management, Institute of Information Management and Finance, National Chiao-Tung University, 1001 Ta Hsueh Road, Hsinchu 300, Taiwan, ROC. E-mail: d88006@csie.ntu.edu.tw. Tel: 886-3-5712121 ext. 57054. Fax: 886-3-5715544. The author was supported in part by MOST grant MOST 103-2410-H-009 -003 -MY3 and NCTU research grant for financial engineering and risk management project.

‡Department of Finance and Department of Computer Science & Information Engineering, National Taiwan University, No. 1, Sec. 4, Roosevelt Road, Taipei 10617, Taiwan. E-mail: lyuu@csie.ntu.edu.tw. The author was supported in part by MOST grant 103-2221-E-002-155-MY3.

numerical methods are developed, including methods related to partial differential equations, lattices, Monte Carlo simulation, and so on [1, 5, 6, 13, 14, 15, 16, 32]. Since the lognormal diffusion process can not satisfactorily capture high-pick and heavy-tail phenomena of stock returns found in many empirical studies [22, 23], recent literature focuses on pricing Asian options based on more generalized price processes, says a Lévy process. Vecer and Xu price Asian options by assuming that the stock price is driven by certain semi-martingale processes [34]. Marena et al. price Asian options with jumps [28]. Bayraktar and Xing consider a jump-diffusion model [3]. Cai and Kou consider a hyper-exponential jump-diffusion model [7]. Vecer derives Black-Scholes-type pricing formulae for the Asian option, and show that the pricing results can be efficiently solved by numerically solving a partial differential equation or a Laplace transform [33]. Note that these recent papers [3, 7, 28, 33, 34] price continuously-sampled Asian options on more generalized processes; however, Asian options with discrete sampling are more popular in financial markets. Ballotta and Kyriakou analyze this pricing problem under the CGMY process with the Monte Carlo simulation [2]. Although the Monte Carlo method is general enough to tackle various price processes, it is relatively inefficient and the result is only probabilistic. Besides, Sesana et al. price various exotic derivatives with numerical quadrature methods [30]. Zhang and Oosterlee price European-style Asian options based on Fourier cosine expansions [35]. Fusai et al. develop the pricing method by maturity randomization [18].

To address the aforementioned pricing problem, this paper improves the error convergence rates of Carverhill and Clewlow's FFT pricing algorithm [9] that can price discretely-sampled Asian options for a large class of price processes. Fusai and Meucci [19] show that this algorithm can be used to price discretely-sampled Asian options under Lévy processes, such as jump-diffusion processes, double exponential processes, and so on. In the Carverhill-Clewlow algorithm, the average stock price is expressed by a recursive formula that involves sums of independent random variables. Thus, the density function of the average stock price can be computed by repeated integral convolutions. Specifically, assume that the stock price is sampled at times $t_0, t_1, t_2, \ldots$. The density function of the average stock price from time t_0 to time t_i can be derived from an integral convolution with the density function of the average stock price from time t_0 to time t_{i-1} and the stock price's transition probability from time t_{i-1} to time t_i, assuming that the density function of the average stock price up to time t_{i-1} and the said transition probability are independent. This independence condition can be satisfied if the stock return process follows Lévy processes. Since the density function of the average stock price can not be expressed analytically, it is approximated by keeping the density values at equidistant grid points in a list in the Carverhill-Clewlow algorithm. Integral convolution is then approximated by discrete convolution, which can be efficiently calculated by the Fast Fourier Transform (FFT). Kept are the density values at $2m+1$ grid points separated by h within a closed interval with a total length of $2mh$ for some integer m. This interval is called a window below. Interpolations are also used in the Carverhill-Clewlow algorithm to approximate functions of a random variable.

Benhamou [4] improves the Carverhill-Clewlow algorithm by narrowing the length of the window. In the Carverhill-Clewlow algorithm, windows should be large enough to contain the bulk of the probability mass of all the density functions. Since the bulk of the probability mass of each density function involved may span different intervals, the length of the window is made large enough to cover all intervals. As a result, the number of grid points, $2m + 1$, and hence the computational time, must be large to keep the pricing error small. In the Benhamou algorithm, the random variables are shifted to make the intervals that contain the bulk of the probability mass of these variables roughly the same, making it more efficient than the Carverhill-Clewlow algorithm, although the asymptotic time complexity remains unchanged.

The first contribution of this paper is a careful theoretical analysis of the aforementioned convolution pricing algorithms, particularly their errors. We prove that both of the Benhamou algorithm and the Carverhill-Clewlow algorithm run in $O(nm \ln m)$ time with the error convergence rate $O(h^2)$, where n denotes the number of stock price samples used in the average stock price (an $O(h^\gamma)$ error convergence rate means the pricing error less than $\tilde{c}h^\gamma$ with a constant $\tilde{c}$.) In both

algorithms, the integral convolutions to evaluate the density values at grid points are approximated by discrete convolutions, which can be calculated by the FFT as they can be rephrased as a multiplication of a Toeplitz matrix and a vector. This multiplication implies that the integral convolution is approximated by the midpoint integration rule, which introduce some errors. From now on, this kind of error is called the integration error. Besides the integration error, there are two other sources of error: the interpolation error and the truncation error. As mentioned before, interpolations are used in both algorithms. This results in the interpolation error. For each interpolation, the error convergence rate depends on the interpolation scheme. For example, it is $O(h^2)$ with the linear interpolation [24]. Finally, the truncation error denotes the pricing error introduced by replacing the infinite support of the density function with a finite one. Černỳ and Kyriakou [10] modify the Carverhill-Clewlow algorithm from a forward-induction algorithm to the backward-induction one and then analyze the resulting truncation error. Benhamou [4] suggests a window size such that the truncation error is negligible. Our numerical results show that with this window, indeed the pricing errors of the Carverhill-Clewlow and Benhamou algorithms are mainly due to the interpolation error and the integration error. Roughly speaking, the quadratic error convergence rates of the Carverhill-Clewlow and Benhamou algorithms are due to the quadratic error convergence rates of the midpoint rule and the linear interpolation.

The second contribution of this paper is an improvement of the Benhamou algorithm. First we reduce the integration error by applying a higher-order Newton-Cotes integration formula. However, it is not a straightforward task because of the constraints on the number of sample points for using a higher-order Newton-Cotes integration formula to approximate a definite integral. For example, Simpson's rule and Boole's rule require $2\kappa+1$ and $4\kappa+1$ sample points respectively, where κ is an integer. To bypass the constraints, we derive new integral rules with higher-order error convergence rates by taking advantage of the Lagrange interpolating polynomial and then combine the Newton-Cotes formula with these integral rules. We show that the integral convolution can be discretely approximated by a multiplication of a Toeplitz matrix and a vector plus a multiplication of a sparse matrix and a vector. The former multiplication can be sped up by the FFT, whereas the latter one can be easily computed due to the sparsity of the matrix. Consequently, the resulting algorithms have a better error convergence rate without increasing the time complexity. In addition, our algorithms retain the flexibility that discretely-sampled Asian options under Lévy processes can be evaluated. Numerical results are given to verify our claims.

This paper is organized as follows. The background knowledge is described in section 2. The Carverhill-Clewlow algorithm and the Benhamou algorithm are also introduced in this section. The quadratic error convergence rates and the time complexities of both algorithms are analyzed in section 3. Section 4 describes how the higher-order Newton-Cotes formulas and the higher-order Newton divided-difference interpolation formulas are used to achieve higher-order error convergence rates without increasing the time complexity. Numerical results in section 5 verify the performance of our approach. Section 6 concludes this paper.

2 Preliminaries

2.1 Financial Background Knowledge

Consider an Asian call option with strike price K initiated at time 0 and maturing at time T. Its underlying stock price at time t is denoted by S_t. The payoff of an Asian call option depends on the average stock price sampled at times t_0, t_1, t_2, $\ldots$, t_n, where $0 = t_0 < t_1 < t_2 < \cdots < t_n = T$. Define the average stock price as

$$A \equiv \frac{1}{n+1} \sum_{i=0}^{n} S_{t_i}.$$

The value of an Asian call option can be expressed as the expected value of the discounted payoff [20]:

$$C = \mathrm{E}\left[e^{-rT}(A-K)^+\right], \tag{1}$$

where r denotes the risk-free interest rate, and $(x)^+$ means $\max(x, 0)$.

The stock return from time t_{i-1} to time t_i, denoted as $R_i \equiv \ln(S_{t_i}/S_{t_{i-1}})$, is assumed to follow a Lévy process. Lévy processes are continuous stochastic processes with stationary independent increments. Specifically, a Lévy process $\{X_t\}_{t\geq 0}$ satisfies the following conditions:

1. $X_0 = 0$ almost surely;
2. $X_t - X_s$ and $X_u - X_t$ are independent for $0 \leq s < t < u < \infty$;
3. for all $0 \leq s < t$, $X_t - X_s$ and X_{t-s} have the same distribution;
4. The paths of $\{X_t\}_{t\geq 0}$ are almost surely right continuous with left limits.

Most models assume the return from time 0 to time t follows a special case of the Lévy process, the Brownian motion:

$$\ln(S_t/S_0) = \left(r - \sigma^2/2\right)t + \sigma W_t,$$

where σ is the volatility of the stock price, and W_t is a standard Brownian motion. Thus the stock price process follows a lognormal diffusion process (LGNO) $S_t = S_0 \exp\{\left(r - \sigma^2/2\right)t + \sigma W_t\}$. Under this assumption, R_i follows a normal distribution with mean $(r - \sigma^2/2)\Delta t_i$ and variance $\sigma^2 \Delta t_i$, where $\Delta t_i \equiv t_i - t_{i-1}$. The LGNO model fails to capture empirical distributions of stock returns, such as high peak, asymmetry and heavy tails. Thus, the more general Lévy processes, like the jump-diffusion model (JD), the double exponential model (DE), the cgmy model[1] (CGMY), and the normal inverse Gaussian model (NIG), have been proposed as alternatives [19]. Indeed, under many Lévy processes, the density functions of R_i do not have closed-form formulas and are approximated by applying the discrete inverse Fourier transform. The characteristic functions of R_i under the aforementioned models are given in Table 1 [19].

2.2 Pricing Algorithms Using the FFT

Define $X_i \equiv e^{R_i} = S_{t_i}/S_{t_{i-1}}$. Carverhill and Clewlow [9] show that

$$\begin{aligned} A &= \frac{1}{n+1}\sum_{i=0}^{n} S_{t_i} = \frac{S_{t_0}}{n+1}\left(1 + X_1 + X_1X_2 + \cdots + X_1X_2\cdots X_n\right) \\ &= \frac{S_{t_0}}{n+1}\left(1 + X_1\left(1 + X_2\left(\cdots X_{n-1}\left(1 + X_n\right)\right)\right)\right) \\ &= \frac{S_{t_0}}{n+1}\left(1 + e^{R_1 + \ln(1+\exp(\cdots + \ln(1+\exp(R_n))))}\right) \\ &= \frac{S_{t_0}}{n+1}\left(1 + e^{B_0}\right), \end{aligned} \tag{2}$$

where the sequence $\{B_i\}$ is defined recursively as follows:

$$\begin{aligned} B_{n-1} &= R_n \\ B_{i-1} &= R_i + \ln(1 + \exp(B_i)) \qquad\qquad i = n-1, n-2, \ldots, 1. \end{aligned} \tag{3}$$

To simplify the notation, we define

$$Y_i \equiv \ln(1 + \exp(B_i)); \tag{4}$$

therefore $B_{i-1} = R_i + Y_i$.

[1] The term cgmy is is named after Carr, P., Geman, H., Madan, D., and Yor, M., the authors of [8].

Let f_X represent the density function of random variable X for convenience. The density function $f_{B_{i-1}}$ equals the convolution of f_{R_i} and f_{Y_i}. Thus $f_{B_{i-1}}$ can be efficiently calculated by applying the inverse Fourier transform on the product of the Fourier transforms of f_{R_i} and f_{Y_i}. Since $\{f_{B_i}\}$ and $\{f_{Y_i}\}$ do not have simple analytical forms, Carverhill and Clewlow estimate them with discrete Fourier and inverse Fourier transforms. Specifically, they choose the window to be $[-c, c]$, which contains the bulk of the probability mass of all densities involved in their algorithm. Then $2m+1$ grid points $-c = x_{-m} < \cdots < x_0 = 0 < \cdots < x_m = c$ are selected in the window with equal grid spacing $h = c/m$. Let $\mathbb{B}_{i,k}$ estimates $f_{B_i}(x_k)$ for $k \in [-m, m]$, and 0 otherwise. The density function f_{B_i} is approximated by the list $\mathbb{B}_i \equiv (\mathbb{B}_{i,-m}, \ldots, \mathbb{B}_{i,0}, \ldots, \mathbb{B}_{i,m})'$. In a similar manner, density estimates for A, R_i and Y_i are stored in lists $\mathbb{A}$, $\mathbb{R}_i$ and $\mathbb{Y}_i$. Thus $f_{B_{i-1}}(x_k)$ can be approximated by a discrete convolution with the midpoint rule as follows:

$$\begin{aligned} f_{B_{i-1}}(x_k) &= \int_{-\infty}^{\infty} f_{R_i}(x_k - x) f_{Y_i}(x) dx \approx \int_{-c-\frac{h}{2}}^{c+\frac{h}{2}} f_{R_i}(x_k - x) f_{Y_i}(x) dx \\ &\approx \sum_{j=-m}^{m} \mathbb{R}_{i,k-j} \mathbb{Y}_{i,j} h. \end{aligned} \tag{5}$$

Define $\mathbb{B}_{i-1,k} \equiv \sum_{j=-m}^{m} \mathbb{R}_{i,k-j} \mathbb{Y}_{i,j} h$ for convenience. Although the use of the midpoint rule in the Carverhill-Clewlow algorithm is not explicitly mentioned in [9], this property is key for its quadratic error convergence rate, as will be shown later. The approximation formula (5) can be succinctly expressed as a multiplication,

$$\mathbb{B}_{i-1} = M_i \mathbb{Y}_i h, \tag{6}$$

where M_i is the following matrix,

$$M_i \equiv \begin{pmatrix} \mathbb{R}_{i,0} & \cdots & \mathbb{R}_{i,-m+1} & \mathbb{R}_{i,-m} & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathbb{R}_{i,m-1} & \cdots & \mathbb{R}_{i,0} & \mathbb{R}_{i,-1} & \mathbb{R}_{i,-2} & \cdots & 0 \\ \mathbb{R}_{i,m} & \cdots & \mathbb{R}_{i,1} & \mathbb{R}_{i,0} & \mathbb{R}_{i,1} & \cdots & \mathbb{R}_{i,-m} \\ 0 & \cdots & \mathbb{R}_{i,2} & \mathbb{R}_{i,1} & \mathbb{R}_{i,0} & \cdots & \mathbb{R}_{i,-m+1} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \mathbb{R}_{i,m} & \mathbb{R}_{i,m-1} & \cdots & \mathbb{R}_{i,0} \end{pmatrix}. \tag{7}$$

Note that M_i is a Toeplitz matrix, that is,

$$M_i(j,k) = M_i(j-1, k-1), \quad j,k = 2,3,\ldots,n, \tag{8}$$

where $M_i(j,k)$ denotes the element located at the i-th row and the j-th column. Obviously, the entrywise product of two Toeplitz matrices is again a Toeplitz matrix and, according to [11], multiplying an $n \times n$ Toeplitz matrix by a $n \times 1$ vector can be efficiently done in time complexity $O(n \log n)$ via the FFT. Both properties will be used in our fast convergent algorithms.

Recall that f_{R_i} is in general not analytically available. To obtain $\mathbb{R}_i$, we apply the discrete inverse Fourier transform on the characteristic function of R_i listed in Table 1, which can be done in $O(m \ln m)$ time by the FFT [19]. In addition, the error convergence rate of this approximation can be enhanced by using the Newton-Cotes formulas. Once $\mathbb{R}_i$ is obtained for all i, we let $\mathbb{B}_{n-1} = \mathbb{R}_n$. $\mathbb{B}_0$ can be evaluated recursively by Eq. (6), where $\mathbb{Y}_i$, the density estimates for Y_i, can be evaluated by applying interpolation to $\mathbb{B}_i$ with

$$f_{Y_i}(x) = \begin{cases} f_{B_i}(\ln(e^x - 1)) \frac{e^x}{e^x - 1}, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}.$$

Algorithm 1 Carverhill-Clewlow algorithm.

1: Evaluate the density values in $\mathbb{B}_{n-1}$ by the density function of $f_{B_{n-1}}$.
2: **for** $i = n-1$ down to 1 **do**
3: Calculate the density values of $\mathbb{Y}_i$ by linear interpolation with $\mathbb{B}_i$.
4: Calculate $\mathbb{B}_{i-1}$ defined in Eq. (6) via the FFT.
5: **end for**
6: Evaluate the option value C by the density function $\mathbb{B}_0$ and Eqs. (1) and (2).

The above equation is derived from the definition of Y_i given by Eq. (4). Finally, with $\mathbb{B}_0$ we can evaluate $\mathbb{A}$ by Eq. (2) and then the option value C by Eq. (1). The Carverhill-Clewlow algorithm is described in Algorithm 1. This paper will show that both the Carverhill-Clewlow algorithm and the Benhamou re-centering algorithm (introduced later) run in $O(nm \ln m)$ with an error convergence rate $O(h^2)$.

According to [4], the bulk of the probability mass of $\mathbb{B}_i$ shifts significantly as i decreases. This point is illustrated in panel (a) of Figure 1. Thus the window size ($= 2c$) must be large enough to keep the bulk of each density function's mass within the window. And m should be large to keep $h = c/m$ (and hence the pricing error) small, which leads to long running time. To address the problem, Benhamou re-centers the density functions of $\{B_i\}$ by introducing a sequence $\{D_i\}$ defined by

$$D_i \equiv B_i - \mu_i, \tag{9}$$

where μ_i approximates $\mathrm{E}[B_i]$ by the following recurrence relation:

$$\begin{aligned} \mu_{n-1} &= E[R_n] \\ \mu_{i-1} &= E[R_i] + \ln(1 + \exp(\mu_i)), \qquad i = n-1, n-2, \ldots, 1. \end{aligned}$$

In consequence, $\{\mathbb{D}_i\}$, defined by the lists that estimate the probability densities $\{f_{D_i}\}$, would not shift as significantly as $\{\mathbb{B}_i\}$, as shown in Figure 1. With re-centering, a window with length $9\sigma\sqrt{nT}$ would suffice, which means $c = 9\sigma\sqrt{nT}/2$. This paper will follow this setting.

In addition, Benhamou [4] defines

$$Z_i \equiv Y_i - \mu_{i-1} = \ln(1 + \exp(D_i + \mu_i)) - \mu_{i-1}, \tag{10}$$

which replaces the role of Y_i in the Carverhill-Clewlow algorithm. As a result, the recurrence relation (3) can be rewritten as

$$D_{n-1} = R_n - \mu_{n-1}, \tag{11}$$

$$D_{i-1} = R_i + Z_i, \qquad i = n-1, n-2, \ldots, 1. \tag{12}$$

The density function f_{D_0} is approximated by the list $\mathbb{D}_0$ derived by the above recurrence relation. With the density function of D_0 and Eq. (2), the price of an Asian option can be expressed as

$$\begin{aligned} C &= e^{-rT} \int_0^\infty f_A(y)(y-K)^+ \, dy = e^{-rT} \int_{-\infty}^\infty f_{D_0}(x) \left(\frac{S_{t_0}}{n+1} \left(1 + e^{x+\mu_0}\right) - K \right)^+ dx \\ &= e^{-rT} \int_\zeta^\infty f_{D_0}(x) \left(\frac{S_{t_0}}{n+1} \left(1 + e^{x+\mu_0}\right) - K \right) dx, \end{aligned} \tag{13}$$

where ζ is the kink point that makes $(S_{t_0}(1 + e^{x+\mu_0})/(n+1) - K)^+$ fail to be smooth as a function of x. Equation (13) can be evaluated by applying the numerical integration on $\mathbb{D}_0$. Algorithm 2 describes the Benhamou algorithm, where the density estimates for f_{Z_i} are stored in the list $\mathbb{Z}_i$. This paper will show that the pricing errors for both the Carverhill-Clewlow algorithm and the Benhamou algorithm converge quadratically.

Algorithm 2 Benhamou algorithm.

1: Evaluate the densities values in the list $\mathbb{D}_{n-1}$.
2: **for** $i = n-1$ down to 1 **do**
3: Calculate $\mathbb{Z}_i$ by linear interpolation with $\mathbb{D}_i$ (see Eq. (10)).
4: Calculate $\mathbb{D}_{i-1}$, the discrete convolution of $\mathbb{R}_i$ and $\mathbb{Z}_i$, via the FFT.
5: **end for**
6: Evaluate the option value C by the density function $\mathbb{D}_0$ and Eqs. (1) and (2).

2.3 Evaluating the Hedging Parameters (Greek letters)

A hedge parameter (or a "Greek letter") measures the changes of the option value with respect to the change of a market variable, say the underlying stock price. Efficiently and correctly evaluating Greek letters is important for risk management. One of the advantages of the above FFT pricing algorithms is that they can be easily extended to evaluate Greek letters as in [19]. For example, the Greek letter delta can be measured as the derivative of the price of an Asian option with respect to the initial stock price S_{t_0}. By differentiating the option value expressed in Eq. (13) with respect to S_{t_0}, we obtain

$$\frac{\partial C}{\partial S_{t_0}} = \frac{e^{-rT}}{n+1} \int_{\zeta}^{\infty} f_{D_0}(x) \left(1 + e^{x+\mu_0}\right) \, dx. \tag{14}$$

This formula can be evaluated by applying the numerical integration on $\mathbb{D}_0$ and is similar to Eq. (13). This entails that the analysis of the error convergence rate for evaluating an option value by Eq. (13) can be applied to the analysis for calculating delta. Similarly, gamma, which can be measured as the second derivative of the option price with respect to S_{t_0}, can be evaluated by an integration formula similar to Eq. (14). This entails that, under the Carverhill-Clewlow algorithm and the Benhamou algorithm, the error convergence rates for calculating delta and gamma are also quadratic. Our revised algorithm that improves the error convergence rate for calculating an option value to a higher order can improve those of delta and gamma to the same order.

2.4 Newton-Cotes Integration Formulas

The Newton-Cotes integration formulas approximate an integral by summing the values of the integrand at equal-spaced points, like the grid points $\{x_i\}$ defined earlier. For example, $\int_a^b g(x)\,dx$ can be approximated by $\int_a^b \phi(x)\,dx$, where $\phi(x)$ denotes an interpolating function for $\{(x_i, g(x_i)\}$. Recall that $h = c/m$ denotes the grid spacing. So $\int_{x_i-h/2}^{x_i+h/2} g(x)\,dx$ can be approximated by setting $\phi(x)$ as a constant function that passes through $(x_i, g(x_i))$ as follows:

$$\int_{x_i-\frac{h}{2}}^{x_i+\frac{h}{2}} g(x)\,dx = hg(x_i) + O(h^3).$$

By summing up the above formula with respect to $i = -m, -m+1, \ldots, m$, the following midpoint rule obtains:

$$\int_{x_{-m}-\frac{h}{2}}^{x_m+\frac{h}{2}} g(x)\,dx = h\sum_{i=-m}^{m} g(x_i) + (2m+1)O(h^3) = h\sum_{i=-m}^{m} g(x_i) + O(h^2) \approx h\sum_{i=-m}^{m} g(x_i).$$

Evaluating the convolutions (3) and (12) implicitly uses the midpoint rule, which is key to our proof that the pricing errors of the Carverhill-Clewlow algorithm and the Benhamou algorithm are both $O(h^2)$. To improve the error convergence rate, higher-order Newton-Cotes integration formulas will be employed. For example, Simpson's rule is derived by using a quadratic polynomial $\phi(x)$ as the interpolation function:

$$\int_{x_i}^{x_{i+2}} g(x)\,dx = \frac{h}{3}(g(x_i) + 4g(x_{i+1}) + g(x_{i+2})) + O(h^5). \tag{15}$$

By summing up for $i = -m, -m+2, \ldots, m-2$, the composite Simpson's rule obtains [24]:

$$\int_{x_{-m}}^{x_m} g(x)\,dx = \frac{h}{3}\left(\sum_{i=-m,-m+2,\ldots,m-2} (g(x_i) + 4g(x_{i+1}) + g(x_{i+2}))\right) + O(h^4). \tag{16}$$

Note that the number of sample points x_{-m}, x_{-m+1}, $\ldots$, x_m must be odd for the composite Simpson's rule. Similarly, approximating $g(x)$ by a quartic interpolating polynomial $\phi(x)$ gives Boole's rule [24]:

$$\begin{aligned}\int_{x_{-m}}^{x_m} g(x)\,dx &= \frac{h}{45}\left(\sum_{i=-m,-m+4,\ldots,m-4} (14g(x_i) + 64g(x_{i+1}) + 24g(x_{i+2})\right. \\ &\quad \left. +64g(x_{i+3}) + 14g(x_{i+4}))\right) + O(h^6).\end{aligned} \tag{17}$$

The number of sample points should be $4\ell + 1$, for some positive integer ℓ for Boole's rule.

3 Analysis of the Carverhill-Clewlow and Benhamou Algorithms

This section will show that the Benhamou algorithm converges at a rate of $O(h^2)$ and its time complexity is $O(nm \ln m)$. As the Carverhill-Clewlow algorithm is a degenerate case of the Benhamou algorithm with $\mu_i \equiv 0$, the same results hold for the Carverhill-Clewlow algorithm.

3.1 Error Convergence Rate

Three different types of errors are introduced by the Benhamou algorithm: the truncation error, the interpolation error and the integration error. The truncation error denotes the error caused by truncating the probability density out of the window $[-c, c]$, which can be neglected when the window is large enough. The interpolation error denotes the error caused by estimating the density values in $\mathbb{Z}_i$ with $\mathbb{D}_i$ by interpolation in line 3 of Algorithm 2. The integration error denotes the error caused by estimating f_{D_i} and f_{R_i} with discrete convolution in line 4 of Algorithm 2. Note that the rate for $\mathbb{R}_i$ to converge to f_{R_i} depends on the kind of Newton-Cotes formula we choose. When the midpoint rule is used, we have

$$|\mathbb{R}_{i,k} - f_{R_i}(x_k)| = O(h^2). \tag{18}$$

Theorem 3.1 *The overall pricing error of the Benhamou algorithm converges to zero at a rate of* $O(h^2)$.

Proof. Recall that the truncation error is negligible. So we will analyze the error contributed by the accumulation of the integration and interpolation errors. First, we prove by induction on i that $\mathbb{D}_i$, the density estimates for D_i, converge to f_{D_i} at a rate of $O(h^2)$. For the base case ($i = n-1$), $\mathbb{D}_{n-1,k}$ stores the approximated value of $f_{D_{n-1}}(x) = f_{R_n}(x_k + E[R_n])$ by interpolating $\mathbb{R}_n$ (see Eq. (11)). Recall that $D_{n-1} = R_n - E[R_n]$. Since the error in $\mathbb{R}_n$ (or $|\mathbb{R}_{n,k} - f_{R_n}(x_k)|$) is $O(h^2)$ by Eq. (18), the error to obtain $\mathbb{D}_{n-1,k}$ (or $|\mathbb{D}_{n-1,k} - f_{D_{n-1}}(x_k)|$) by interpolating $\mathbb{R}_{n-1}$ is $O(h^2)$ [24]. The inductive step will show that $|\mathbb{D}_{i-1,k} - f_{D_{i-1}}(x_k)| = O(h^2)$ under the premise that

$$|\mathbb{D}_{i,k} - f_{D_i}(x_k)| = O(h^2). \tag{19}$$

By the triangle inequality,

$$\begin{aligned}
\left|\mathbb{D}_{i-1,k} - f_{D_{i-1}}(x_k)\right| &= \left|\sum_{j=-m}^{m} \mathbb{Z}_{i,j}\mathbb{R}_{i,k-j}h - f_{D_{i-1}}(x_k)\right| \\
&\leq \sum_{j=-m}^{m} \left|\mathbb{Z}_{i,j} - f_{Z_i}(x_j)\right| \mathbb{R}_{i,k-j}h + \left|\sum_{j=-m}^{m} f_{Z_i}(x_j)\mathbb{R}_{i,k-j}h - f_{D_{i-1}}(x_k)\right| \\
&\leq \underbrace{\sum_{j=-m}^{m} \left|\mathbb{Z}_{i,j} - f_{Z_i}(x_j)\right| \mathbb{R}_{i,k-j}h}_{\mathbf{A}} + \underbrace{\sum_{j=-m}^{m} f_{Z_i}(x_j) \left|\mathbb{R}_{i,k-j} - f_{R_i}(x_{k-j})\right| h}_{\mathbf{B}} \\
&\quad + \underbrace{\left|\sum_{j=-m}^{m} f_{Z_i}(x_j) f_{R_i}(x_{k-j})h - \int_{x_{-m}-\frac{h}{2}}^{x_m+\frac{h}{2}} f_{Z_i}(x) f_{R_i}(x_k - x)\, dx\right|}_{\mathbf{C}} \\
&\quad + \underbrace{\left|\int_{x_{-m}-\frac{h}{2}}^{x_m+\frac{h}{2}} f_{Z_i}(x) f_{R_i}(x_k - x)\, dx - f_{D_{i-1}}(x_k)\right|}_{\mathbf{D}}.
\end{aligned}$$

For expression **B**, we have $|\mathbb{R}_{i,k-j} - f_{R_i}(x_{k-j})| = O(h^2)$ by Eq. (18) and

$$\sum_{j=-m}^{m} f_{Z_i}(x_j)h = 1 + O(h^2) \tag{20}$$

since $\sum_{j=-m}^{m} f_{Z_i}(x_j)h$ is a truncated approximation for $\int_{-\infty}^{\infty} f_{Z_i}(x)dx = 1$ by the midpoint rule. Thus $\mathbf{B} = O(h^2)$.

For expression **A**, we have

$$\sum_{j=-m}^{m} f_{R_i}(x_{k-j})h = 1 + O(h^2)$$

by the similar arguments leading to Eq. (20). Therefore,

$$\begin{aligned}
\sum_{j=-m}^{m} \mathbb{R}_{i,k-j}h &\leq \sum_{j=-m}^{m} f_{R_i}(x_{k-j})h + \sum_{j=-m}^{m} \left|\mathbb{R}_{i,k-j} - f_{R_i}(x_{k-j})\right| h \\
&= 1 + O(h^2) + (2m+1)\, O(h^3) = 1 + O(h^2).
\end{aligned} \tag{21}$$

$\mathbb{Z}_{i,j}$ is obtained by applying linear interpolation on Eq. (10) with the density values from $\mathbb{D}_i$. The error to obtain $\mathbb{Z}_{i,j}$ by interpolating $\mathbb{D}_i$, i.e., $|\mathbb{Z}_{i,j} - f_{Z_i}(x_j)|$, is $O(h^2)$ given Eq. (19) [24]. Substituting this and Eq. (21) into expression **A**, we have $\mathbf{A} = O(h^2)(1 + O(h^2)) = O(h^2)$. Expression **D** can be ignored since the truncation error can be neglected by taking a large window. Expression **C** denotes the integration error of the midpoint rule, which is $O(h^2)$. To sum up, $\left|\mathbb{D}_{i-1,k} - f_{D_{i-1}}(x_k)\right| = O(h^2)$. Thus, by induction, we obtain

$$|\mathbb{D}_{0,k} - f_{D_0}(x_k)| = O(h^2). \tag{22}$$

Finally, we proceed to derive the option price $\int_0^{\infty} f_A(x)(x-K)^+ dx$ in terms of f_{D_0} and show that the pricing error of the Benhamou algorithm is $O(h^2)$. The relationship between random variables A and D_0 can be derived by Eqs. (2) and (9) as follows:

$$A = \frac{S_{t_0}}{n+1}(1 + e^{D_0+\mu_0}).$$

Thus the option value can be rewritten by changing variables as follows:

$$\begin{aligned}\int_0^\infty f_A(y)(y-K)^+\,dy &= \int_{-\infty}^{\infty} f_{D_0}(x)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x+\mu_0}\right)-K\right)^+ dx \\ &= \int_{\ln\left(\frac{K(n+1)}{S_{t_0}}-1\right)-\mu_0}^{\infty} f_{D_0}(x)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x+\mu_0}\right)-K\right) dx \qquad (24)\end{aligned}$$

For convenience, we divide the above integral into the principal term

$$\int_{x_{k_0}-\frac{h}{2}}^{\infty} f_{D_0}(x)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x+\mu_0}\right)-K\right) dx, \qquad (25)$$

and the remainder term

$$\int_{\ln\left(\frac{K(n+1)}{S_{t_0}}-1\right)-\mu_0}^{x_{k_0}-\frac{h}{2}} f_{D_0}(x)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x+\mu_0}\right)-K\right) dx, \qquad (26)$$

where k_0 is the smallest integer such that $x_{k_0}-\frac{h}{2} \geq \ln\left(\frac{K(n+1)}{S_{t_0}}-1\right)-\mu_0$. The principal term (25) is easy to approximate by applying the midpoint rule with the grid list $\mathbb{D}_0$. The approximation error can be shown to be $O(h^2)$ as follows:

$$\begin{aligned}&\left|\int_{x_{k_0}-\frac{h}{2}}^{\infty} f_{D_0}(x)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x+\mu_0}\right)-K\right) dx - h\sum_{k=k_0}^{m}\mathbb{D}_{0,k}\left(\frac{S_{t_0}}{n+1}\left(1+e^{x_k+\mu_0}\right)-K\right)\right| \\ \leq\ &\left|\int_{x_{k_0}-\frac{h}{2}}^{\infty} f_{D_0}(x)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x+\mu_0}\right)-K\right) dx - \int_{x_{k_0}-\frac{h}{2}}^{x_m+\frac{h}{2}} f_{D_0}(x)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x+\mu_0}\right)-K\right) dx\right| \qquad (27) \\ +\ &\left|\int_{x_{k_0}-\frac{h}{2}}^{x_m+\frac{h}{2}} f_{D_0}(x)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x+\mu_0}\right)-K\right) dx - h\sum_{k=k_0}^{m} f_{D_0}(x_k)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x_k+\mu_0}\right)-K\right)\right| \qquad (28) \\ +\ &\left|h\sum_{k=k_0}^{m} f_{D_0}(x_k)\left(\frac{S_{t_0}}{n+1}\left(1+e^{x_k+\mu_0}\right)-K\right) - h\sum_{k=k_0}^{m}\mathbb{D}_{0,k}\left(\frac{S_{t_0}}{n+1}\left(1+e^{x_k+\mu_0}\right)-K\right)\right|.\end{aligned}$$

Since the truncation error in Eq. (27) can be neglected by picking a large window and the integration error of the midpoint rule (28) is $O(h^2)$, the RHS of the above inequality is at most

$$\begin{aligned}&O(h^2)+h\sum_{k=k_0}^{m}\left|f_{D_0}(x_k)-\mathbb{D}_{0,k}\right|\left(\frac{S_{t_0}}{n+1}\left(1+e^{x_k+\mu_0}\right)-K\right) \\ =\ &O(h^2)+hO(h^2)\sum_{k=k_0}^{m}\left(\frac{S_{t_0}}{n+1}\left(1+e^{x_k+\mu_0}\right)-K\right) \qquad (29) \\ \leq\ &O(h^2)+hO(h^2)m\left(\frac{S_{t_0}}{n+1}\left(1+e^{x_m+\mu_0}\right)-K\right) \qquad (30) \\ =\ &O(h^2)\end{aligned}$$

since $m=O(1/h)$, where Eq. (29) is based on Eq. (22) and $((1+e^{x_m+\mu_0})-K)$ is a constant as $x_m=c$. On the other hand, the remainder term (26) is simply an integral of a smooth function over a closed interval with length less than h, and the integrand equals zero at the integral's lower limit. This remainder term is bounded by $O(h^2)$ as shown in Appendix A. Thus, the error introduced by neglecting the remainder term is $O(h^2)$. Consequently, the Benhamou algorithm converges quadratically. □

In our algorithms, the convergence rates of the interpolation and integration errors will be improved by higher-order Newton-Cotes formulas and the higher-order Newton divided-difference interpolation formula.

3.2 Time Complexity

Theorem 3.2 *The time complexity of the Benhamou algorithm is* $O(nm \ln m)$.

Proof. Note that $\{\mathbb{R}_i\}$ $(1 \le i \le n)$ is evaluated by the inverse Fourier transform on the characteristic function of R_i in $O(nm \ln m)$ time. Then $\mathbb{D}_{n-1}$ is evaluated in $O(m)$ time by applying interpolation on $\mathbb{R}_{n-1}$. Next, $\mathbb{D}_0$ is evaluated by alternately repeating interpolations and convolutions $n-1$ times. $\mathbb{Z}_i$ is evaluated by applying interpolation on $\mathbb{D}_i$, which takes $O(m)$ time. Convoluting $\mathbb{R}_i$ and $\mathbb{Z}_i$ takes $O(m \ln m)$ time. Thus it takes $O((n-1)(m+m \ln m)) = O(nm \ln m)$ time to evaluate $\mathbb{D}_0$. The option value in Eq. (1) can be evaluated by numerical integration with $\mathbb{D}_0$ in $O(m)$ time. To sum up, the total time complexity is $O(nm \ln m)$ time. □

Note that the time complexity of the Carverhill-Clewlow algorithm is also $O(nm \ln m)$ and can be analyzed with a proof similar to the above. The next section will show that our algorithms can achieve faster error convergence rate with the same asymptotic time complexity.

4 Our Fast Convergent Algorithms

We will first demonstrate how the error convergence rate can be improved from $O(h^2)$ to $O(h^4)$ by reducing the integration error and the interpolation error to $O(h^4)$ by our modified Simpson's rule and the third-order Newton divided-difference interpolation formula, respectively. The time complexity remains $O(nm \ln m)$. Then we show that the error convergence rate can be further improved by using the higher-order Newton-Cotes formula and the Newton divided-difference interpolation formula.

4.1 Algorithms with Higher-order Convergence Rates

First we try to improve the integration error due to convolution.[2] In the Benhamou algorithm, evaluating the grid list $\mathbb{D}_i$ by convolution can be reduced to matrix multiplication $\mathbb{D}_{i-1} = M_i \mathbb{Z}_i h$, where M_i is a Toeplitz matrix in Eq. (6). Note that this computation can be sped up by the FFT. As shown earlier, $M_i \mathbb{Z}_i h$ can be thought of as applying $2m+1$ midpoint rules. To improve the error convergence rate, we use higher-order Newton-Cotes formulas such as Simpson's rule to replace the midpoint rule. However, this idea may encounter two problems. First, since Simpson's rule requires an odd number of sample points, not all of the $2m+1$ integrations can be directly approximated by it. Hence some modifications of Simpson's rule are needed. Second, the matrix M_i is no longer Toeplitz after the modifications, and the computation of $M_i \mathbb{Z}_i h$ can no longer be sped up by the FFT. Fortunately, the modified M_i can be decomposed into the sum of a Toeplitz matrix and a sparse matrix. As a result, the time complexity for computing $M_i \mathbb{Z}_i h$ remains unchanged, as we will show later.

Now we describe how to use the modified Simpson's rule to satisfy the requirement on the number of sample points. To estimate $f_{D_{i-1}}(x_k)$ that involves an odd number of sample points with the constraint $x_k < 0$ (i.e., k is a negative even number), Simpson's rule is directly applied to obtain

$$\begin{aligned} f_{D_{i-1}}(x_k) &= \int_{-\infty}^{\infty} f_{R_i}(x_k - x) f_{Z_i}(x)\, dx \\ &\approx \int_{x_{-m}}^{x_{m+k}} f_{R_i}(x_k - x) f_{Z_i}(x)\, dx \qquad (31) \\ &\approx h \sum_{j=-m}^{m+k} w_j \mathbb{R}_{i,k-j} \mathbb{Z}_{i,j} \\ &\equiv \mathbb{D}_{i-1,k}. \end{aligned}$$

[2]Note that the error $|\mathbb{R}_{i,k} - f_{R_i}(x_k)|$ due to the discrete inverse Fourier transform can be improved by using the Newton-Cotes formulas.

where the range of integration is $x_{-m} < x < x_{k+m}$ to make both $x_k - x$ and x in Eq. (31) remain in the window, and $(w_j)_{j=-m}^{m+k}, (\hat{w}_j)_{j=-m+1}^{m+k}$ and $(\tilde{w}_j)_{j=k-m}^{m}$ denote Simpson's coefficients (but note their different subscripts): $\left(\frac{1}{3}, \frac{4}{3}, \frac{2}{3}, \frac{4}{3}, \ldots, \frac{4}{3}, \frac{1}{3}\right)$. Similarly, if $x_k > 0$ (i.e., k is a positive even number), we have

$$\begin{aligned} f_{D_{i-1}}(x_k) &\approx \int_{x_{k-m}}^{x_m} f_{R_i}(x_k - x) f_{Z_i}(x)\, dx \\ &\approx h \sum_{j=k-m}^{m} \tilde{w}_j \mathbb{R}_{i,k-j} \mathbb{Z}_{i,j} \\ &\equiv \mathbb{D}_{i-1,k}. \end{aligned}$$

On the other hand, if an even number of sample points (i.e., k is odd) are involved in the numerical integration, we apply a modified integration rule with a shorter integration interval

$$\int_{x_j}^{x_j+h} g(x)\, dx \approx \frac{5g(x_j) + 8g(x_j + h) - g(x_j + 2h)}{12} h \tag{32}$$

and its dual version

$$\int_{x_j}^{x_j+h} g(x)\, dx \approx \frac{-g(x_j - h) + 8g(x_j) + 5g(x_j + h)}{12} h.$$

These integration formulas can be derived by integrating the Lagrange interpolating formula of $g(x)$, and this quadrature can be proved to have the same error convergence rate $O(h^4)$ as Simpson's rule (see Appendix A). Numerical integrations with an even number of sample points (i.e., k is odd) can now be solved by combining Simpson's rule and the modified integration rule (32). If k is a negative odd number (i.e., an even number of sample points with $x_k < 0$), we have

$$\begin{aligned} f_{D_{i-1}}(x_k) &\approx \int_{x_{-m}}^{x_{m+k}} f_{R_i}(x_k - x) f_{Z_i}(x)\, dx \\ &= \int_{x_{-m}}^{x_{-m+1}} f_{R_i}(x_k - x) f_{Z_i}(x)\, dx + \int_{x_{-m+1}}^{x_{m+k}} f_{R_i}(x_k - x) f_{Z_i}(x)\, dx \\ &\approx \frac{5\mathbb{R}_{i,k+m}\mathbb{Z}_{i,-m} + 8\mathbb{R}_{i,k+m-1}\mathbb{Z}_{i,-m+1} - \mathbb{R}_{i,k+m-2}\mathbb{Z}_{i,-m+2}}{12} h + h \sum_{j=-m+1}^{m+k} \hat{w}_j \mathbb{R}_{i,k-j} \mathbb{Z}_{i,j} \\ &\equiv \mathbb{D}_{i-1,k}, \end{aligned}$$

where we use the modified integration rule (32) and Simpson's rule, and $\hat{w}_j$ denotes Simpson's coefficients. Similarly, if $x_k > 0$ (i.e., k is a positive odd number), we have

$$\begin{aligned} f_{D_{i-1}}(x_k) &\approx \int_{x_{k-m}}^{x_m} f_{R_i}(x_k - x) f_{Z_i}(x)\, dx \\ &= \int_{x_{k-m}}^{x_{m-1}} f_{R_i}(x_k - x) f_{Z_i}(x)\, dx + \int_{x_{m-1}}^{x_m} f_{R_i}(x_k - x) f_{Z_i}(x)\, dx \\ &\approx h \sum_{j=k-m}^{m-1} \tilde{w}_j \mathbb{R}_{i,k-j} \mathbb{Z}_{i,j} + \frac{-\mathbb{R}_{i,k-m+2}\mathbb{Z}_{i,m-2} + 8\mathbb{R}_{i,k-m+1}\mathbb{Z}_{i,m-1} + 5\mathbb{R}_{i,k-m}\mathbb{Z}_{i,m}}{12} h \\ &\equiv \mathbb{D}_{i-1,k}. \end{aligned}$$

Note that in any case $|f_{D_{i-1}}(x_k) - \mathbb{D}_{i-1,k}|$ converges at a rate of $O(h^4)$ since both the modified integration rule Eq. (32) and Simpson's rule do. By combining the above results, the convergence rate of the integration error can be improved by replacing $\mathbb{D}_{i-1} = M_i \mathbb{Z}_i h$ with $\mathbb{D}_i = \left(M^{\text{Simpson}} * M_i\right) \mathbb{Z}_{i+1} h$,

where "$*$" stands for the entrywise product and M^{Simpson} is given by

$$\begin{pmatrix}
\frac{1}{3} & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{1}{3} & 0 & \cdots & 0 & 0 & 0 & 0 & 0 \\
\frac{5}{12} & \frac{8}{12}+\frac{1}{3} & -\frac{1}{12}+\frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{1}{3} & \cdots & 0 & 0 & 0 & 0 & 0 \\
\vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\
\frac{1}{3} & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \cdots & \frac{1}{3} & 0 & 0 & 0 & 0 \\
\frac{5}{12} & \frac{8}{12}+\frac{1}{3} & -\frac{1}{12}+\frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{1}{3} & 0 & 0 & 0 \\
\frac{1}{3} & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{1}{3} & 0 & 0 \\
\frac{5}{12} & \frac{8}{12}+\frac{1}{3} & -\frac{1}{12}+\frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \frac{1}{3} & 0 \\
\frac{1}{3} & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \frac{1}{3} \\
0 & \frac{1}{3} & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{2}{3} & \frac{4}{3}-\frac{1}{12} & \frac{1}{3}+\frac{8}{12} & \frac{5}{12} \\
0 & 0 & \frac{1}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \frac{1}{3} \\
0 & 0 & 0 & \frac{1}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{2}{3} & \frac{4}{3}-\frac{1}{12} & \frac{1}{3}+\frac{8}{12} & \frac{5}{12} \\
0 & 0 & 0 & 0 & \frac{1}{3} & \cdots & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \frac{1}{3} \\
\vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\
0 & 0 & 0 & 0 & 0 & \cdots & \frac{1}{3} & \frac{4}{3} & \frac{2}{3} & \cdots & \frac{4}{3} & \frac{2}{3} & \frac{4}{3}-\frac{1}{12} & \frac{1}{3}+\frac{8}{12} & \frac{5}{12} \\
0 & 0 & 0 & 0 & 0 & \cdots & 0 & \frac{1}{3} & \frac{4}{3} & \cdots & \frac{2}{3} & \frac{4}{3} & \frac{2}{3} & \frac{4}{3} & \frac{1}{3}
\end{pmatrix}.$$

Because $M^{\text{Simpson}} * M_i$ is not a Toeplitz matrix, $\mathbb{D}_i = \left(M^{\text{Simpson}} * M_i\right) \mathbb{Z}_{i+1} h$ can not be evaluated by the FFT. To address this problem, we decompose M^{Simpson} into the sum of M^{Toeplitz} and $M^{\text{remainder}}$, where

$$M^{\text{Toeplitz}} \equiv \frac{1}{3}\begin{pmatrix}
2 & 4 & 2 & 4 & 2 & \cdots & 4 & 1 & 0 & \cdots & 0 & 0 & 0 & 0 & 0 \\
4 & 2 & 4 & 2 & 4 & \cdots & 2 & 4 & 1 & \cdots & 0 & 0 & 0 & 0 & 0 \\
\vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\
2 & 4 & 2 & 4 & 2 & \cdots & 4 & 2 & 4 & \cdots & 1 & 0 & 0 & 0 & 0 \\
4 & 2 & 4 & 2 & 4 & \cdots & 2 & 4 & 2 & \cdots & 4 & 1 & 0 & 0 & 0 \\
2 & 4 & 2 & 4 & 2 & \cdots & 4 & 2 & 4 & \cdots & 2 & 4 & 1 & 0 & 0 \\
4 & 2 & 4 & 2 & 4 & \cdots & 2 & 4 & 2 & \cdots & 4 & 2 & 4 & 1 & 0 \\
1 & 4 & 2 & 4 & 2 & \cdots & 4 & 2 & 4 & \cdots & 2 & 4 & 2 & 4 & 1 \\
0 & 1 & 4 & 2 & 4 & \cdots & 2 & 4 & 2 & \cdots & 4 & 2 & 4 & 2 & 4 \\
0 & 0 & 1 & 4 & 2 & \cdots & 4 & 2 & 4 & \cdots & 2 & 4 & 2 & 4 & 2 \\
0 & 0 & 0 & 1 & 4 & \cdots & 2 & 4 & 2 & \cdots & 4 & 2 & 4 & 2 & 4 \\
0 & 0 & 0 & 0 & 1 & \cdots & 4 & 2 & 4 & \cdots & 2 & 4 & 2 & 4 & 2 \\
\vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\
0 & 0 & 0 & 0 & 0 & \cdots & 1 & 4 & 2 & \cdots & 4 & 2 & 4 & 2 & 4 \\
0 & 0 & 0 & 0 & 0 & \cdots & 0 & 1 & 4 & \cdots & 2 & 4 & 2 & 4 & 2
\end{pmatrix}$$

and

$$M^{\text{remainder}} \equiv \begin{pmatrix}
-\frac{1}{3} & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & 0 \\
-\frac{11}{12} & \frac{1}{3} & -\frac{1}{12} & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & 0 \\
\vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\
-\frac{1}{3} & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & 0 \\
-\frac{11}{12} & \frac{1}{3} & -\frac{1}{12} & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & 0 \\
-\frac{1}{3} & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & 0 \\
-\frac{11}{12} & \frac{1}{3} & -\frac{1}{12} & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & -\frac{1}{12} & \frac{1}{3} & -\frac{11}{12} \\
0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & -\frac{1}{3} \\
0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & -\frac{1}{12} & \frac{1}{3} & -\frac{11}{12} \\
0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & -\frac{1}{3} \\
\vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\
0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & -\frac{1}{12} & \frac{1}{3} & -\frac{11}{12} \\
0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & -\frac{1}{3}
\end{pmatrix}.$$

Since both the entrywise product and the standard matrix product satisfy the distribution law with matrix addition, we have the following identity:

$$\begin{aligned}
\mathbb{D}_i &= \left(M^{\text{Simpson}} * M_i\right) \mathbb{Z}_{i+1} h \\
&= \left(\left(M^{\text{Toeplitz}} + M^{\text{remainder}}\right) * M_i\right) \mathbb{Z}_{i+1} h \\
&= \left(M^{\text{Toeplitz}} * M_i + M^{\text{remainder}} * M_i\right) \mathbb{Z}_{i+1} h \\
&= \left(M^{\text{Toeplitz}} * M_i\right) \mathbb{Z}_{i+1} h + \left(M^{\text{remainder}} * M_i\right) \mathbb{Z}_{i+1} h. \qquad (33)
\end{aligned}$$

Therefore, the calculation of $\mathbb{D}_i$ can be decomposed into two parts, $\left(M^{\text{Toeplitz}} * M_i\right) \mathbb{Z}_{i+1} h$ and $\left(M^{\text{remainder}} * M_i\right) \mathbb{Z}_{i+1} h$. For the first part, since $M^{\text{Toeplitz}} * M_i$ is a Toeplitz matrix, $\left(M^{\text{Toeplitz}} * M_i\right) \mathbb{Z}_i h$ can be calculated with time complexity $O(m \ln m)$ via the FFT. For the second part, there are at most 3 non-zero entries in each row of the sparse matrix $M^{\text{remainder}}$ (hence $M^{\text{remainder}} * M_i$, too), so $\left(M^{\text{remainder}} * M_i\right) \mathbb{Z}_i h$ can be evaluated in $O(m)$ time. To sum up, we have constructed a discrete convolution algorithm that converges to the integral convolution at a rate of $O(h^4)$ with time complexity $O(m \ln m)$.

4.2 Improving the Convergence Rate of the Interpolation Error

The order of the error convergence rate for the interpolation used in line 3 of Algorithm 2 can be improved by the Newton divided-difference interpolation formula [24]. From the definition of Z_i in Eq. (10), we have

$$f_{Z_i}(x) = \begin{cases} f_{D_i}(\ln(e^{x+\mu_{i-1}} - 1) - \mu_i) \frac{e^{x+\mu_{i-1}}}{e^{x+\mu_{i-1}} - 1} & \text{if } x > -\mu_{i-1} \\ 0 & \text{otherwise} \end{cases}. \qquad (34)$$

To calculate $\mathbb{Z}_{i,k}$, the approximation of $f_{Z_i}(x_k)$, we first find the corresponding density value $f_{D_i}(\ln(e^{x_k+\mu_{i-1}} - 1) - \mu_i)$. This density value is obtained by interpolating $\mathbb{D}_i$. To make the interpolation error converges at a rate of $O(h^4)$, we first select four grid points $\{x_j, \ldots, x_{j+3}\}$ such that $(\ln(e^{x+\mu_{i-1}} - 1) - \mu_i) \in [x_j, x_{j+3}]$, and then derive a cubic interpolation polynomial $\rho(x)$ that passes through $\{(x_j, \mathbb{D}_{i,j}), \ldots, (x_{j+3}, \mathbb{D}_{i,j+3})\}$. The density value $\mathbb{Z}_{i,k}$ is then calculated by

$$\mathbb{Z}_{i,k} = \begin{cases} \rho(\ln(e^{x_k+\mu_{i-1}} - 1) - \mu_i) \frac{e^{x_k+\mu_{i-1}}}{e^{x_k+\mu_{i-1}} - 1} & \text{if } x_k > -\mu_{i-1} \\ 0 & \text{otherwise} \end{cases}. \qquad (35)$$

4.3 Improving the Error Convergence Rate of the Pricing Results

The 4th-order approximation to the density values f_{D_0} at grid points $x_{-m}, x_{-m+1}, \ldots, x_m$ in the previous subsection can be used to evaluate the option value Eq. (24) by numerical integration. Note that the lower limit $\ln\left(\frac{K(n+1)}{S_{t_0}} - 1\right) - \mu_0$ in the integral may no be in the grid list, making Simpson's rule not directly applicable to Eq. (24).

To address this problem, we divide Eq. (24) into the principal term

$$\int_{x_{k^*}}^{\infty} f_{D_0}(x) \left(\frac{S_{t_0}}{n+1} \left(1 + e^{x+\mu_0}\right) - K \right) dx \tag{36}$$

and the remainder term

$$\int_{\ln\left(\frac{K(n+1)}{S_{t_0}}-1\right)-\mu_0}^{x_{k^*}} f_{D_0}(x) \left(\frac{S_{t_0}}{n+1} \left(1 + e^{x+\mu_0}\right) - K \right) dx, \tag{37}$$

where k^* stands for the smallest integer such that $x_{k^*} \geq \ln\left(\frac{K(n+1)}{S_{t_0}} - 1\right) - \mu_0$. Note that Eq. (24) is the sum of Eqs. (36) and (37). The lower limit in Eq. (36) is x_{k^*} instead of $x_{k_0} - h/2$ (see Eq. (25)). This is because we will use Simpson's rule to reduce the integration error, and the settings of lower and upper limits of Simpson's rule and the midpoint rule used in Eq. (25) are different. By ignoring the truncation error, the principal term can be approximated as follows:

$$\int_{x_{k^*}}^{\infty} f_{D_0}(x) \left(\frac{S_{t_0}}{n+1} \left(1 + e^{x+\mu_0}\right) - K \right) dx \approx \int_{x_{k^*}}^{x_m} f_{D_0}(x) \left(\frac{S_{t_0}}{n+1} \left(1 + e^{x+\mu_0}\right) - K \right) dx, \tag{38}$$

where the right-hand side can be approximated by Simpson's rule and our modified integration rule of Eq. (32). With the 4th-order approximation for f_{D_0} at grid points, applying Simpson's rule, the principal term is approximated with error $O(h^4)$. As for the remainder term, we use the definite integral of an interpolating function as an approximation. Specifically, we use the Lagrange interpolating polynomial mentioned in Appendix A to derive a quadratic polynomial $\varphi(x)$ that passes through

$$\left(\ln\left(\frac{K(n+1)}{S_{t_0}} - 1\right) - \mu_0, 0 \right),$$
$$\left(x_{k^*}, \mathbb{D}_{0,k^*} \left(\frac{S_{t_0}}{n+1} \left(1 + e^{x_{k^*}+\mu_0}\right) - K \right) \right),$$
$$\left(x_{k^*+1}, \mathbb{D}_{0,k^*+1} \left(\frac{S_{t_0}}{n+1} \left(1 + e^{x_{k^*+1}+\mu_0}\right) - K \right) \right),$$

and then use

$$\int_{\ln\left(\frac{K(n+1)}{S_{t_0}}-1\right)-\mu_0}^{x_{k^*}} \varphi(x)\, dx \tag{39}$$

to approximate the remainder term. This approximation has an error bound $O(h^4)$ as shown in Appendix A. Combining the above, we obtain a 4th-order approximation for the option value.

4.4 Time Complexity of Our Algorithms

$\{\mathbb{R}_i\}$ $(1 \leq i \leq n)$ can be evaluated by the higher-order inverse Fourier transform in $O(nm \ln m)$ time. Each element in list $\mathbb{D}_{n-1}$ is evaluated by applying higher-order interpolation on $\mathbb{R}_{n-1}$ in constant time [24]. Thus, it takes $O(m)$ time to evaluate $\mathbb{D}_{n-1}$. Next, $\mathbb{D}_0$ is evaluated by repeating interpolations and convolutions $n-1$ times. Evaluating $2m+1$ elements in $\mathbb{Z}_i$ by the higher-order interpolation takes $O(m)$ time and convoluting $\mathbb{R}_i$ and $\mathbb{Z}_i$ with our convolution mentioned in the last subsection takes $O(m \ln m)$ time. Thus the time complexity to evaluate $\mathbb{D}_0$ is $O(n(m + m \ln m)) = O(nm \ln m)$. Finally, the option value defined in Eq. (1) can be evaluated by the Newton-Cotes integration formulas with $\mathbb{D}_0$ in $O(m)$ time. Thus the total time complexity is $O(nm \ln m)$ time.

4.5 Higher-Order Converging Algorithms

Earlier, Simpson's rule (16) and the modified integration rules (32) are used together with the cubic polynomial interpolation formula to improve the error convergence rate from $O(h^2)$ to $O(h^4)$. The error convergence rate can be further improved by applying higher order Newton-Cotes formulas and Newton divided-difference interpolation formulas. For example, Boole's rule and the quintic polynomial interpolation formula can be applied to improve the error convergence rate to $O(h^6)$. Recall that Boole's rule uses the integral of a quartic polynomial to approximate the desired integration with an error convergence rate $O(h^6)$ (see Eq. (17)). Since the number of sample points must be $4\ell + 1$ under Boole's rule for an integer ℓ, some modified integration rules similar to Eq. (32) are needed. The integration rules we use are listed below, all of which converge at a rate of $O(h^6)$. The derivation and the error bound analysis are given in Appendix A.

$$\int_{x_j}^{x_j+3h} g(x)\,dx = \frac{27g(x_j) + 102g(x_j+h) + 72g(x_j+2h) + 42g(x_j+3h) - 3g(x_j+4h)}{80}h \quad (40)$$

$$\int_{x_j}^{x_j+2h} g(x)\,dx = \frac{29g(x_j) + 128g(x_j+h) + 24g(x_j+2h) + 4g(x_j+3h) - g(x_j+4h)}{90}h \quad (41)$$

$$\int_{x_j}^{x_j+h} g(x)\,dx = \frac{251g(x_j) + 646g(x_j+h) - 264g(x_j+2h) + 106g(x_j+3h) - 19g(x_j+4h)}{720}h. \quad (42)$$

Similarly, the dual version of the the aforementioned integration formulas are listed

$$\int_{x_j+h}^{x_j+4h} g(x)\,dx = \frac{-3g(x_j) + 42g(x_j+h) + 72g(x_j+2h) + 102g(x_j+3h) + 27g(x_j+4h)}{80}h$$

$$\int_{x_j+2h}^{x_j+4h} g(x)\,dx = \frac{-g(x_j) + 4g(x_j+h) + 24g(x_j+2h) + 128g(x_j+3h) + 29g(x_j+4h)}{90}h$$

$$\int_{x_j+3h}^{x_j+4h} g(x)\,dx = \frac{-19g(x_j) + 106g(x_j+h) - 264g(x_j+2h) + 646g(x_j+3h) + 251g(x_j+4h)}{720}h.$$

Thus the approximated value of $f_{D_{i-1}}(x_k)$, i.e., $\mathbb{D}_{i-1,k}$, can be calculated with Boole's rule and the aforementioned modified integration rule by mimicking the idea to combine the integration rules proposed in subsection 4.1. For example, $f_{D_{i-1}}(x_{-3})$ can be approximated as follows:

$$\begin{aligned}
& f_{D_{i-1}}(x_{-3}) = \int_{-\infty}^{\infty} f_{R_i}(x_{-3}-x) f_{Z_i}(x)\,dx \approx \int_{x_{-m}}^{x_{m-3}} f_{R_i}(x_{-3}-x) f_{Z_i}(x)\,dx \\
= & \int_{x_{-m}}^{x_{-m+1}} f_{R_i}(x_{-3}-x) f_{Z_i}(x)\,dx + \sum_{j=-m+1,-m+5,\ldots,m-7} \int_{x_j}^{x_{j+4}} f_{R_i}(x_{-3}-x) f_{Z_i}(x)\,dx \\
\approx & \frac{h}{720}(251\mathbb{R}_{i,m-3}\mathbb{Z}_{i,-m} + 646\mathbb{R}_{i,m-4}\mathbb{Z}_{i,-m+1} - 264\mathbb{R}_{i,m-5}\mathbb{Z}_{i,-m+2} + 106\mathbb{R}_{i,m-6}\mathbb{Z}_{i,-m+3} - 19\mathbb{R}_{i,m-7}\mathbb{Z}_{i,-m+4}) \quad (43) \\
+ & \sum_{j=-m+1,-m+5,\ldots,m-7} \frac{h}{45}(14\mathbb{R}_{i,j-3}\mathbb{Z}_{i,j} + 64\mathbb{R}_{i,j-4}\mathbb{Z}_{i,j+1} + 24\mathbb{R}_{i,j-5}\mathbb{Z}_{i,j+2} + 64\mathbb{R}_{i,j-6}\mathbb{Z}_{i,j+3} + 14\mathbb{R}_{i,j-7}\mathbb{Z}_{i,j+4}) \quad (44) \\
\equiv & \mathbb{D}_{i-1,-3},
\end{aligned}$$

where Eq. (42) is used in Eq. (43), and Boole's rule is used in Eq. (44). Replacing Simpson's rule by Boole's rule in Eq. (33), and modified integration rule Eq. (32) by Eq. (42), the vector $\mathbb{D}_{i-1}$ can be more accurately evaluated. As a result, we obtain $\mathbb{D}_{i-1} = (M^{\text{Boole}} * M)\mathbb{Z}_i h$, where M^{Boole}

is defined by

$$\begin{pmatrix}
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
\frac{251}{720} & \frac{646}{720}+\frac{14}{45} & \frac{-264}{720}+\frac{64}{45} & \frac{106}{720}+\frac{24}{45} & \frac{-19}{720}+\frac{64}{45} & \frac{28}{45} & \cdots & \frac{24}{45} & \frac{64}{45} & \frac{14}{45} & 0 & 0 & 0 \\
\frac{29}{90} & \frac{128}{90} & \frac{24}{90}+\frac{14}{45} & \frac{4}{90}+\frac{64}{45} & \frac{-1}{90}+\frac{24}{45} & \frac{64}{45} & \cdots & \frac{64}{45} & \frac{24}{45} & \frac{64}{45} & \frac{14}{45} & 0 & 0 \\
\frac{27}{80} & \frac{102}{80} & \frac{72}{80} & \frac{42}{80}+\frac{14}{45} & \frac{-3}{80}+\frac{64}{45} & \frac{24}{45} & \cdots & \frac{28}{45} & \frac{64}{45} & \frac{24}{45} & \frac{64}{45} & \frac{14}{45} & 0 \\
\frac{14}{45} & \frac{64}{45} & \frac{24}{45} & \frac{64}{45} & \frac{28}{45} & \frac{64}{45} & \cdots & \frac{64}{45} & \frac{28}{45} & \frac{64}{45} & \frac{24}{45} & \frac{64}{45} & \frac{14}{45} \\
0 & \frac{14}{45} & \frac{64}{45} & \frac{24}{45} & \frac{64}{45} & \frac{28}{45} & \cdots & \frac{28}{45} & \frac{64}{45}+\frac{-3}{80} & \frac{14}{45}+\frac{42}{80} & \frac{72}{80} & \frac{102}{80} & \frac{27}{80} \\
0 & 0 & \frac{14}{45} & \frac{64}{45} & \frac{24}{45} & \frac{64}{45} & \cdots & \frac{64}{45} & \frac{24}{45}+\frac{-1}{90} & \frac{64}{45}+\frac{4}{90} & \frac{14}{45}+\frac{24}{90} & \frac{128}{90} & \frac{29}{90} \\
0 & 0 & 0 & \frac{14}{45} & \frac{64}{45} & \frac{24}{45} & \cdots & \frac{28}{45} & \frac{64}{45}+\frac{-19}{720} & \frac{24}{45}+\frac{106}{720} & \frac{64}{45}+\frac{-264}{720} & \frac{14}{45}+\frac{646}{720} & \frac{251}{720} \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots
\end{pmatrix}.$$

Note that the first row contains the coefficients used for evaluating $\mathbb{D}_{i-1,-3}$ (see Eqs. (43) and (44)). To speed up the evaluation with the FFT, we decompose $M^{\text{Boole}} = M^{\text{Toeplitz}} + M^{\text{remainder}}$, where

$$M^{\text{Toeplitz}} \equiv \frac{1}{45}\begin{pmatrix}
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
64 & 28 & 64 & 24 & 64 & 28 & \cdots & 24 & 64 & 14 & 0 & 0 & 0 \\
24 & 64 & 28 & 64 & 24 & 64 & \cdots & 64 & 24 & 64 & 14 & 0 & 0 \\
64 & 24 & 64 & 28 & 64 & 24 & \cdots & 28 & 64 & 24 & 64 & 14 & 0 \\
14 & 64 & 24 & 64 & 28 & 64 & \cdots & 64 & 28 & 64 & 24 & 64 & 14 \\
0 & 14 & 64 & 24 & 64 & 28 & \cdots & 24 & 64 & 28 & 64 & 24 & 64 \\
0 & 0 & 14 & 64 & 24 & 64 & \cdots & 64 & 24 & 64 & 28 & 64 & 24 \\
0 & 0 & 0 & 14 & 64 & 24 & \cdots & 28 & 64 & 24 & 64 & 28 & 64 \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots
\end{pmatrix}$$

is a Toeplitz matrix, and

$$M^{\text{remainder}} \equiv \begin{pmatrix}
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
\frac{251}{720} & \frac{646}{720} & -\frac{264}{720} & \frac{106}{720} & -\frac{19}{720} & 0 & \cdots & 0 & 0 & 0 & 0 & 0 & 0 \\
\frac{29}{90} & \frac{128}{90} & \frac{24}{90} & \frac{4}{90} & -\frac{1}{90} & 0 & \cdots & 0 & 0 & 0 & 0 & 0 & 0 \\
\frac{27}{80} & \frac{102}{80} & \frac{72}{80} & \frac{42}{80} & -\frac{3}{80} & 0 & \cdots & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 & -\frac{3}{80} & \frac{42}{80} & \frac{72}{80} & \frac{102}{80} & \frac{27}{80} \\
0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 & -\frac{1}{90} & \frac{4}{90} & \frac{24}{90} & \frac{128}{90} & \frac{29}{90} \\
0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 & -\frac{19}{720} & \frac{106}{720} & -\frac{264}{720} & \frac{646}{720} & \frac{251}{720} \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots
\end{pmatrix}$$

has non-zero values only at the first and last five columns. Recall from Eq. (33) that the vector $\mathbb{D}_{i-1}$ is the sum of two matrix multiplications. Since $M^{\text{Toeplitz}} * M_i$ is also a Toeplitz matrix, $(M^{\text{Toeplitz}} * M_i)\mathbb{Z}_{i+1}h$ can be calculated in $O(m \ln m)$ time via the FFT. On the other hand,

$(M^{\text{remainder}} * M_i)\mathbb{Z}_{i+1}h$ can be calculated in $O(m)$ time since there are only $O(m)$ non-zero entries in $M^{\text{remainder}}$. To sum up, the vector $\mathbb{D}_{i-1}$ can be evaluated in $O(m \ln m)$ time. Note that the convergence rate of the integration error is $O(h^6)$ since the error convergence rates of Boole's rule and the modified integration rules Eq. (40), (41) and (42) are both $O(h^6)$. Similarly, the convergence rate of the interpolation error can be improved to $O(h^6)$ simply by substituting a quintic interpolation polynomial $\rho(x)$ (instead of a cubic one) into Eq. (35). By repeatedly applying the aforementioned convolution method and the interpolation method, we can obtain the 6th-order approximation to f_{D_0} in $O(nm \ln m)$ time. To ensure that the pricing error also converges in $O(h^6)$, we first recall that the option value in Eq. (24) can be decomposed into the principal term in Eq. (36) and the remainder term in Eq. (37). The principal term Eq. (36) can be rewritten as Eq. (38) and then approximated by Boole's rule and the integration rules Eq. (40), (41) or (42). To estimate the remainder term Eq. (37), we first derive a quartic interpolating polynomial $\varphi(x)$ that passes through $\left(\ln\left(\frac{K(n+1)}{S_{t_0}} - 1\right) - \mu_0, 0\right)$, $\left(x_{k^*}, \mathbb{D}_{0,k^*}\left(\frac{S_{t_0}}{n+1}\left(1 + e^{x_{k^*}+\mu_0}\right) - K\right)\right)$, $\left(x_{k^*+1}, \mathbb{D}_{0,k^*+1}\left(\frac{S_{t_0}}{n+1}\left(1 + e^{x_{k^*+1}+\mu_0}\right) - K\right)\right)$, $\left(x_{k^*+2}, \mathbb{D}_{0,k^*+2}\left(\frac{S_{t_0}}{n+1}\left(1 + e^{x_{k^*+2}+\mu_0}\right) - K\right)\right)$, and $\left(x_{k^*+3}, \mathbb{D}_{0,k^*+3}\left(\frac{S_{t_0}}{n+1}\left(1 + e^{x_{k^*+3}+\mu_0}\right) - K\right)\right)$ by the Lagrange interpolating polynomial mentioned in Appendix A. Then the remainder term is approximated by integrating $\varphi(x)$ as in Eq (39). Since the error convergence rates of these two approximations are $O(h^6)$, the pricing error of our method converges at a rate of $O(h^6)$.

5 Numerical Results

To compare the error convergence rates among the pricing algorithms in this paper, the log-log plots on the absolute pricing errors against grid spacing h are employed. We first plot the points $(\ln h, \ln(e(h)))$ of that algorithm with various h. The estimated order of error convergence rate γ is the slope of the linear regression line for these points.

If the stock price follows LGNO and the strike price $K = 0$, the Asian call option can be priced analytically [12]. In this case we can calculate exact pricing errors and compare the error convergence rate between the Benhamou algorithm and our $O(h^4)$ and $O(h^6)$ algorithms in panel (a) of Figure 2. The slopes of the linear regression functions are 2.07429, 4.2457, and 6.13852, respectively, which implies that the error convergent rates for these three algorithms are roughly $O(h^2)$, $O(h^4)$ and $O(h^6)$, respectively.

Analytical formulas are unavailable when the strike price is not zero, however. Figure 3 shows that the pricing results of Benhamou algorithm and our $O(h^4)$ and $O(h^6)$ algorithms converge to the same value as the number of grid points m tends to infinity. Panel (a) plots the pricing results of the three algorithms. Note that the Benhamou algorithm (denoted by circles) converges much more slowly than the other two algorithms. The difference between our 4th-order and 6th-order algorithms can not be clearly observed since the scale for the y-axis is too coarse. A finer scale is used in panel (b), which suggests that the pricing results of our 6th-order algorithm (denoted by squares) converge faster than our 4th-order algorithm (denoted by triangles). The plot for $\ln h$ against the natural logarithm of the absolute pricing error is illustrated in panel (b) of Figure 2, where the option value obtained by using the extrapolation method proposed in [21] on the numerical results in Figure 3 is used as a proxy of the exact value. The slopes of the linear regression functions for the three algorithms are 207217, 4.24851, 6.03232, respectively, which again implies that the error convergence rates for the three algorithms are $O(h^2)$, $O(h^4)$ and $O(h^6)$, respectively. In addition, as mentioned in Sec. 2.3, our algorithms can be easily extended to evaluate delta with a higher order of error convergence rate than that of the Benhamou algorithm. Numerical results for calculating delta (defined in Eq. (14)) illustrated in Figure 4 suggest that our 6th-order algorithm converges faster than the 4th-order one. Both algorithms converge much faster than the Benhamou algorithm.

When n is large enough, our algorithms also converge to the price of a continuously-sampled Asian option. The price of a continuously-sampled Asian option has been widely studied, and

accurate pricing results under the LGNO model can be obtained (see, for example, [29, 36]). We use the pricing results suggested by [36] as the benchmarks and compare them with our 4th-order algorithm in Table 2. As shown in the table, for all parameter settings, the pricing results generated by our algorithms approximate the benchmarks very well.

Our algorithms also converge more smoothly in pricing Asian options under various Lévy processes than the Benhamou one. The pricing results under the NIG, the DE, the JD, and the CGMY models are illustrated in Figure 5. While the pricing results generated by the Benhamou algorithm (plotted in circles) oscillate significantly, the results of ours (plotted in squares) converge smoothly.

The running times for the aforementioned three algorithms are compared in Figure 6, where the x-axis and y-axis denote the natural logarithm of absolute pricing error and the running time, respectively. Clearly, given the same running time, our 4th-order algorithm always generates more accurate results than the Benhamou algorithm. Thus we conclude that our approach does significantly improve the efficiency.

6 Conclusion

Asian options can be priced numerically by estimating the density of the payoff function with repeated interpolation and convolution. Previous algorithms use the midpoint rule to approximate the convolution, which can be sped up by the FFT. We show that the pricing error of their algorithms converges quadratically. To improve the error convergence rate, we reduce the integration error by higher-order Newton-Cotes integration formulas, like Simpson's rule and Boole's rule, and the interpolation error by higher-order Newton divided-difference interpolation. We also derive the modified integration rule to bypass the constraint on the number of sample points required by the higher-order Newton-Cotes formulas. The resulting convolution can be sped up by the FFT. Thus our algorithms can achieve the same time complexity as previously announced algorithms, but at a faster error convergence rate. Numerical experiments show the superiority of our algorithms to price Asian options under some Lévy processes that are frequently used in the literature.

References

[1] D. Aingworth, R. Motwani, and J.D. Oldham. Accurate approximations for Asian options. In *Proceedings of the 11th ACM-SIAM SODA*, pages 891–900, 2000.

[2] L. Ballotta and I. Kyriakou. Monte Carlo simulation of the cgmy process and option pricing. forthcoming in. *Journal of Futures Markets*, 2014.

[3] E. Bayraktar and H. Xing. Pricing Asian options for jump diffusion. *Mathematical Finance*, 21(1):117–143, 2011.

[4] E. Benhamou. Fast Fourier transform for discrete Asian options. *Journal of Computational Finance*, 6(1):49–68, 2002.

[5] P. Boyle, M. Broadie, and P. Glasserman. Monte Carlo methods for security pricing. *Journal of Economic Dynamics and Control*, 21(8–9):1267–1321, 1997.

[6] M. Broadie, P. Glasserman, and S.G. Kou. Connecting discrete and continuous path-dependent options. *Finance and Stochastics*, 3(1):55–82, 1999.

[7] N. Cai and S. Kou. Pricing Asian options under a hyper-exponential jump diffusion model. *Operations Research*, 60(1):64–77, 2012.

[8] P. Carr, H. Geman, D. Madan, and M. Yor. The fine structure of asset returns: an empirical investigation. *Journal of Business*, 75(2):305–332, 2002.

[9] A. Carverhill and L. Clewlow. Flexible convolution. *Risk*, 3(4):25–29, 1990.

[10] A. Černỳ and I. Kyriakou. An improved convolution algorithm for discretely sampled Asian options. *Quantitative Finance*, 11(3):381–389, 2011.

[11] T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein. *Introduction to algorithms, 2nd ed.* Cambridge, MA: MIT Press, 2001.

[12] T.S. Dai, G.S. Huang, and Y.D. Lyuu. An efficient convergent lattice algorithm for European Asian options. *Applied Mathematics and Computation*, 169(2):1458–1471, 2005.

[13] T.S. Dai and Y.D. Lyuu. Efficient, exact algorithms for Asian options with multiresolution lattices. *Review of Derivatives Research*, 5:181–203, 2002.

[14] T.S. Dai and Y.D. Lyuu. An exact subexponential-time lattice algorithm for Asian options. *Acta Informatica*, 44(1):23–39, 2007.

[15] T.S. Dai and Y.D. Lyuu. Accurate and efficient lattice algorithms for American-style Asian options with range bounds. *Applied Mathematics and Computation*, 209:238–253, 2009.

[16] T.S. Dai, J.Y. Wang, and H.S. Wei. Adaptive placement method on pricing arithmetic average options . *Review of Derivatives Research*, 11:83–118, 2008.

[17] M.C. Fu, D.B. Dilip, and T. Wang. Pricing continuous Asian options: a comparison of Monte Carlo and Laplace transform inversion methods. *Journal of Computational Finance*, 2:49–74, 1999.

[18] G. Fusai, D. Marazzina, and M. Marena. Pricing discretely monitored Asian options by maturity randomization. *SIAM Journal on Financial Mathematics*, 2(1):383–403, 2011.

[19] G. Fusai and A. Meucci. Pricing discretely monitored Asian options under Lévy processes. *Journal of Banking & Finance*, 32(10):2076–2088, 2008.

[20] J.M. Harrison and S.R. Pliska. Martingales and stochastic integrals in the theory of continuous trading. *Stochastic Processes and Their Applications*, 11(3):215–260, 1981.

[21] S. Heston and G. Zhou. On the rate of convergence of discrete-time contingent claims. *Mathematical Finance*, 10(1):53–75, 2000.

[22] J. Hosking, G. Bonti, and D. Siegel. Beyond the lognormal. *Risk*, 13(5):59–62, 2000.

[23] R. Huisman, K.G. Koedijk, and R.A.J. Pownall. VaR-x: Fat tails in financial risk management. *Journal of Risk*, 1(1):47–61, 1998.

[24] E. Isaacson and H.B. Keller. *Analysis of numerical methods*. New York, Dover, 1994.

[25] A.G.Z. Kemna and A.C.F. Vorst. A pricing method for options based on average asset values. *Journal of Banking & Finance*, 14(1):113–129, 1990.

[26] E. Levy. Pricing European average rate currency options. *Journal of International Money and Finance*, 11(5):474–491, 1992.

[27] M. Marena, G. Fusai, and G. Longo. *Handbook of Multi-Commodity Markets and Products: Structuring, Trading, and Risk Management*, chapter Asian options in commodity markets: structuring, pricing, and hedging. John Wiley & Sons, 2014.

[28] M. Marena, A. Roncoroni, and G. Fusai. Asian options with jumps. *Argo Newsletter New Frontiers in Practical Risk Management*, 1(1):53–62, 2013.

[29] M.A. Milevsky and S.E. Posner. Asian options, the sum of lognormals, and the reciprocal gamma distribution. *Journal of Financial and Quantitative Analysis*, 33(2):409–422, 1998.

[30] D. Sesana, D. Marazzina, and G. Fusai. Pricing exotic derivatives exploiting structure. *European Journal of Operational Research*, 236(1):369–381, 2014.

[31] S. Turnbull and L.M. Wakeman. A quick algorithm for pricing European average options. *Journal of Financial and Quantitative Analysis*, 26(3):377–389, 1991.

[32] J. Vecer. A new pde approach for pricing arithmetic average Asian options. *Journal of Computational Finance*, 4(4):105–113, 2001.

[33] J. Vecer. Black-scholes representation for Asian options. forthcoming in. *Mathematical Finance*, 2012.

[34] M. Xu and J. Vecer. Pricing Asian options in a semimartingale model. *Quantitative Finance*, 4(2):170–175, 2004.

[35] B. Zhang and C.W. Oosterlee. Efficient pricing of European-style Asian options under exponential Lévy processes based on Fourier cosine expansions. *SIAM Journal on Financial Mathematics*, 4(1):399–426, 2013.

[36] J.E. Zhang. Pricing continuously sampled Asian options with perturbation method. *Journal of Futures Markets*, 23(6):535–560, 2003.

[37] P.G. Zhang. *Exotic options, 2nd ed.* Singapore, World Scientific, 1998.

A Derivation of General Numerical Integration Formulas Given the Error Convergence Rates

To integrate a smooth but unknown function $g(t)$ given the function's values at some points, we first derive an approximating polynomial $\phi(t)$ by the Lagrange interpolating polynomial and then derive the numerical integration formula by integrating $\phi(t)$ instead. The numerical integration can be reinterpreted as the weighted average values of $g(t)$ at the points and the error convergence rate can be represented as a function of the number of points and the spacing between the points. A general form of this numerical integration formula will be first introduced. Then we will show how Eq. (32) is derived and how Eq. (39) is integrated numerically by this general form. Finally, we will show that the pricing error for neglecting the remainder term in the Benhamou algorithm (see subsection 3.1) converges quadratically.

Given the values of $g(t)$ at arbitrary $r-1$ points $t_1 < t_2 < \cdots < t_{r-1}$, the definite integral of $g(t)$ over a given interval $[t_i, t_j] \subseteq [t_1, t_{r-1}]$ can be approximated by the integral of the polynomial $\phi(t)$ that pass through $(t_1, g(t_1)), (t_2, g(t_2)), \ldots, (t_{r-1}, g(t_{r-1}))$. This $\phi(t)$ is the Lagrange interpolating polynomial:

$$\phi(t) = \sum_{s=1}^{r-1} g(t_s) L_s(t),$$

where $L_s(t)$ is the Lagrange basis polynomial defined as follows:

$$L_s(t) = \prod_{l=1, l\neq s}^{r-1} \frac{t - t_l}{t_s - t_l}. \tag{45}$$

As $t \in [t_1, t_{k-1}]$, the error between $g(t)$ and $\phi(t)$ is bounded above by $O(k^{r-1})$[24], where $k = \max_{2\leq l\leq r-1}(t_l - t_{l-1})$. That is, $|\phi(t) - g(t)| \leq ck^{r-1}$. Taking integral of both $g(t)$ and $\phi(t)$, we know the error is

$$\left| \int_{t_i}^{t_j} \phi(t)dt - \int_{t_i}^{t_j} g(t)dt \right| \leq \int_{t_i}^{t_j} |\phi(t) - g(t)|\, dt \leq \int_{t_i}^{t_j} ck^{r-1} dt = c(t_j - t_i)k^{r-1} = O(k^r).$$

Furthermore, since $\phi(t)$ is a linear combination of the Lagrange basis polynomials, the integration of $\phi(t)$ can be rewritten as the weighted average values of $g(t)$ as follows:

$$\int_{t_i}^{t_j} \phi(t)dt = \int_{t_i}^{t_j} \left(\sum_{s=1}^{r-1} g(t_s) L_s(t) \right) dt = \sum_{s=1}^{r-1} \left(\int_{t_i}^{t_j} L_s(t)dt \right) g(t_s). \tag{46}$$

Two special examples are given below to demonstrate how the aforementioned general forms are used to derive numerical integration formulas. We first consider the case when $t_1, t_2, \ldots, t_{r-1}$ are equidistant. Suppose that we want to derive an integration formula with an error convergence rate $O(k^4)$ as in Eq. (32). We first substitute $r = 4$ and $t_3 - t_2 = t_2 - t_1 = k$ into Eq. (45) to obtain

$$\begin{aligned} L_1(t) &= \frac{(t - t_2)(t - t_3)}{(t_1 - t_2)(t_1 - t_3)} = \frac{(t - t_1 - k)(t - t_1 - 2k)}{2k^2}, \\ L_2(t) &= \frac{(t - t_3)(t - t_1)}{(t_2 - t_3)(t_2 - t_1)} = \frac{(t - t_1 - 2k)(t - t_1)}{-k^2}, \\ L_3(t) &= \frac{(t - t_1)(t - t_2)}{(t_3 - t_1)(t_3 - t_2)} = \frac{(t - t_1)(t - t_1 - k)}{2k^2}. \end{aligned} \tag{47}$$

Then we substitute the above identities into Eq. (46). For example,

$$\int_{t_1}^{t_1+k} L_1(t)dt = \int_0^k \frac{(u - k)(u - 2k)}{2k^2} du = \frac{5k}{12},$$

where we use the substitution $u = t - t_1$ in Eq. (47). Other coefficients can also be evaluated by a similar technique. As a result,

$$\sum_{s=1}^{3}\left(\int_{t_1}^{t_1+k} L_s(t)dt\right) g(t_s) = \frac{5g(t_1)+8g(t_2)-g(t_3)}{12}k$$

and

$$\sum_{s=1}^{3}\left(\int_{t_3-k}^{t_3} L_s(t)dt\right) g(t_s) = \frac{-g(t_1)+8g(t_2)+5g(t_3)}{12}k,$$

which yields Eq. (32). Other integration formulas in section 4 can also be derived by the same approach.

Consider the case that the spacing between adjacent grid points are not equal, like the integration for the remainder term in Eq. (39). Let k and $\overline{k}$ stand for $t_3 - t_2$ and $t_2 - t_1$, respectively. Note that $k > \overline{k}$. To ensure an $O(k^4)$ error convergence rate, we substitute $r = 4$, k, and $\overline{k}$ into Eqs. (45) and (46) to obtain

$$\sum_{s=1}^{3}\left(\int_{t_1}^{t_2} L_s(t)dt\right) g(t_s) = \frac{k\left(\overline{k}(3\overline{k}+2k)g(t_1) + \left(3\overline{k}^2+4k\overline{k}+k^2\right)g(t_2) - k^2 g(t_3)\right)}{6\overline{k}(\overline{k}+k)}.$$

Thus Eq. (39) can be numerically solved by substituting $t_1 = \ln\left(\frac{K(n+1)}{S_{t_0}} - 1\right) - \mu_0, t_2 = x_{k^*}, t_3 = x_{k^*+1}$ and $g(t_1) = 0, g(t_2) = \mathbb{D}_{0,k^*}\left(\frac{S_{t_0}}{n+1}\left(1+e^{x_{k^*}+\mu_0}\right) - K\right), g(t_3) = \mathbb{D}_{0,k^*+1}\left(\frac{S_{t_0}}{n+1}\left(1+e^{x_{k^*+1}+\mu_0}\right) - K\right)$ into the above formula.

The aforementioned analysis can be used to show that the pricing error caused by neglecting the remainder term (see Eq. (26)) in the Benhamou algorithm converges quadratically. Substituting $r = 3$ and $\overline{k}$ into Eqs. (45) and (46) yields the Trapezoidal rule:

$$\sum_{s=1}^{2}\left(\int_{t_1}^{t_2} L_s(t)dt\right) g(t_s) = \frac{g(t_1)+g(t_2)}{2}\overline{k},$$

which is an 3rd-order approximation for $\int_{t_1}^{t_2} g(t)dt$. Note that $g(t_1) = 0$ and $g(t_2)$ can be expressed as $g(t_1) + \overline{k}g'(\xi)$ by Taylor expansion, where $\xi \in [t_1, t_2]$. We have

$$\int_{t_1}^{t_2} g(t)\,dt = \frac{g(t_1)+g(t_2)}{2}\overline{k} + O(\overline{k}^3) = \frac{g'(\xi)}{2}\overline{k}^2 + O(\overline{k}^3).$$

Assume that

$$\begin{aligned} t_1 &= \ln\left(\frac{K(n+1)}{S_{t_0}} - 1\right) - \mu_0, \\ t_2 &= x_{k_0} - \frac{h}{2} = t_1 + \bar{k}, \\ g(t) &= f_{D_0}(t)\left(\frac{S_{t_0}}{n+1}\left(1+e^{t+\mu_0}\right) - K\right). \end{aligned}$$

Thus $\int_{t_1}^{t_2} g(t)dt$ is reduced to the remainder term given in Eq. (26), which can now be rewritten as

$$\begin{aligned} & \frac{g(t_1)+g(t_2)}{2}\bar{k} + O(\bar{k}^3) \\ = & \frac{0 + f_{D_0}\left(x_{k_0} - \frac{h}{2}\right)\left(\frac{S_{t_0}}{n+1}\left(1+e^{t_1+\bar{k}+\mu_0}\right) - K\right)}{2}\bar{k} + O(\bar{k}^3) \\ = & \frac{f_{D_0}\left(x_{k_0} - \frac{h}{2}\right)}{2}\left(\frac{S_{t_0}}{n+1}\left(1+e^{t_1+\mu_0}(1+O(\bar{k}))\right) - K\right)\bar{k} + O(\bar{k}^3) \end{aligned}$$

Since $\frac{S_{t_0}}{n+1}\left(1+e^{t_1+\mu_0}\right)-K=0$, the above expression can be reduced to

$$\begin{aligned}&\frac{f_{D_0}\left(x_{k_0}-\frac{h}{2}\right)}{2}\left(\frac{S_{t_0}O(\bar{k})e^{t_1+\mu_0}}{n+1}\right)\bar{k}+O(\bar{k}^3)\\ =\;&\frac{f_{D_0}\left(x_{k_0}-\frac{h}{2}\right)}{2}\left(\frac{S_{t_0}e^{t_1+\mu_0}}{n+1}\right)O(\bar{k}^2)+O(\bar{k}^3).\end{aligned}$$

Substituting $t_1=\ln\left(\frac{K(n+1)}{S_{t_0}}-1\right)-\mu_0$ again into the above expression we obtain

$$\frac{f_{D_0}\left(x_{k_0}-\frac{h}{2}\right)}{2}\left(K-\frac{S_{t_0}}{n+1}\right)O(\bar{k}^2)+O(\bar{k}^3)=O(\bar{k}^2),$$

provided $\frac{f_{D_0}\left(x_{k_0}-\frac{h}{2}\right)}{2}\left(K-\frac{S_{t_0}}{n+1}\right)$ is bounded from above.

Model	Parameters	Characteristic functions $E(e^{\mathbf{i}uR_i})$
LGNO	r, σ	$\exp\left\{\mathbf{i}u\left(r-\frac{\sigma^2}{2}\right)\Delta t_i-\frac{\sigma^2}{2}u^2\Delta t_i\right\}$
JD	$r, \sigma, \lambda, \alpha, \beta$	$\exp\left\{\mathbf{i}u\left(r-\lambda\left(e^{\alpha+\frac{\beta^2}{2}}-1\right)-\frac{\sigma^2}{2}\right)\Delta t_i-\frac{\sigma^2u^2}{2}\Delta t_i+\lambda\Delta t_i\left(e^{i\alpha u-\frac{1}{2}\beta^2u^2}-1\right)\right\}$
DE	$r, \sigma, \lambda, \eta_1, \eta_2, p$	$\exp\left\{\mathbf{i}u\left(r-\lambda\left(\frac{(1-p)\eta_2}{\eta_2+1}+\frac{p\eta_1}{\eta_1-1}-1\right)-\frac{\sigma^2}{2}\right)\Delta t_i-\frac{\sigma^2u^2}{2}\Delta t_i+\lambda\Delta t_i\left(\frac{(1-p)\eta_2}{\eta_2+\mathbf{i}u}+\frac{p\eta_1}{\eta_1-\mathbf{i}u}-1\right)\right\}$
NIG	r, δ, α, β	$\exp\left\{\mathbf{i}u\left(r+\delta\left(\sqrt{\alpha^2-(\beta+1)^2}-\sqrt{\alpha^2-\beta^2}\right)\right)\Delta t_i-\delta\Delta t_i\left(\sqrt{\alpha^2-(\beta+\mathbf{i}u)^2}-\sqrt{\alpha^2-\beta^2}\right)\right\}$
CGMY	C, G, M, Y	$\exp\{\mathbf{i}u\left(r-C\Gamma(-Y)\left((M-1)^Y-M^Y+(G+1)^Y-G^Y\right)\right)t$ $+Ct\Gamma(-Y)\left((M-\mathbf{i}u)^Y-M^Y+(G+\mathbf{i}u)^Y-G^Y\right)\}$

Table 1: The characteristic functions of the stock return under different Lévy processes.

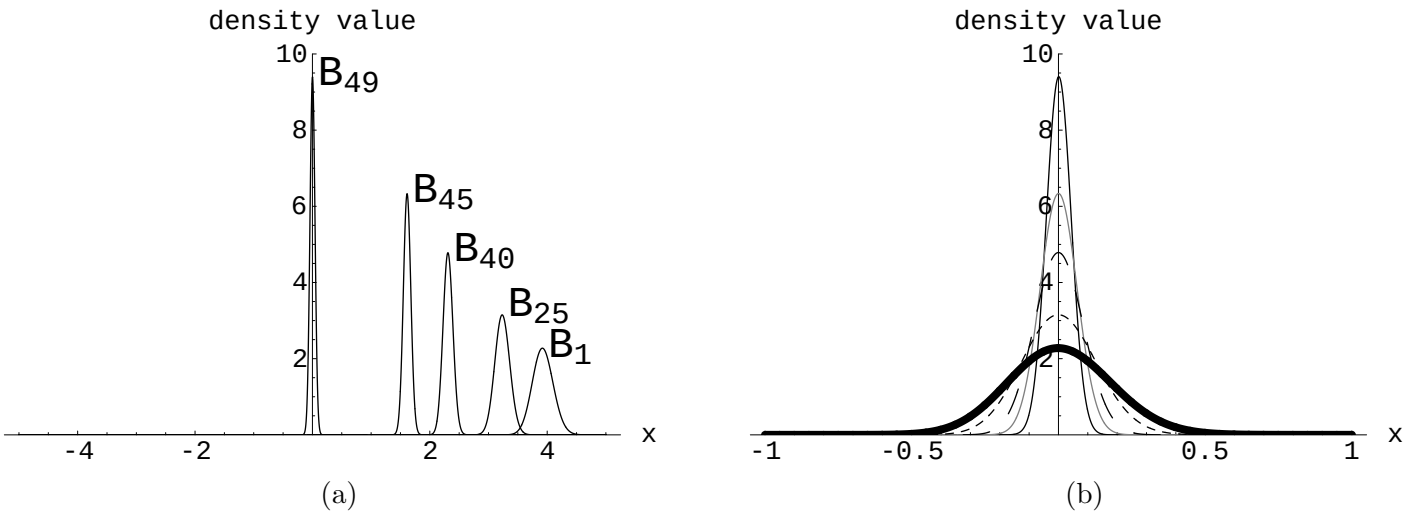
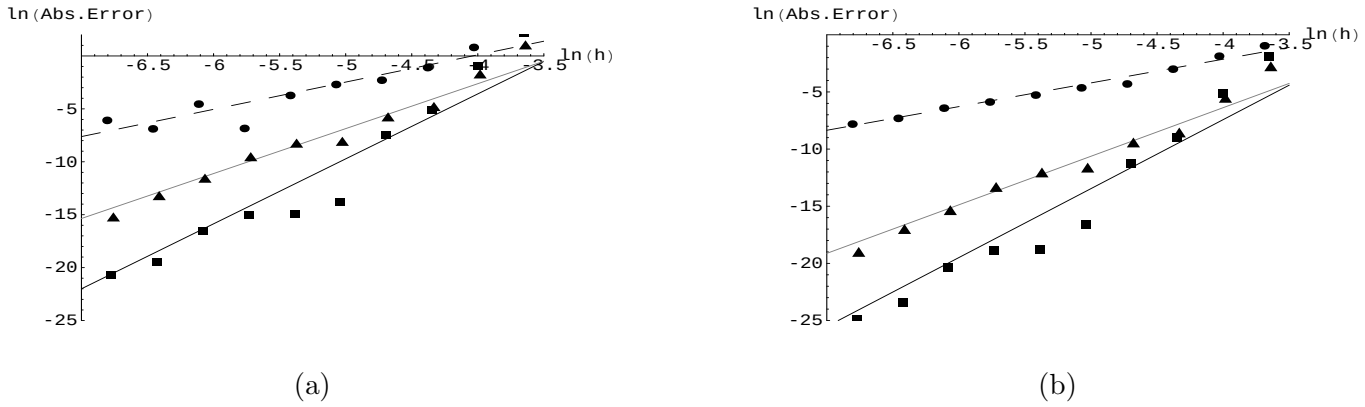


Figure 1: The Plot Densities before and after Re-centering.
Panel (a) denotes the densities of B_i (approximated by $\mathbb{B}_i$) used in the Carverhill-Clewlow algorithm. Panel (b) denotes the densities of D_i (approximated by $\mathbb{D}_i$) used in the Benhamou algorithm. The thin black line, the thin gray line, the long dash line, the short dash line, and the thick black line denote the densities of D_{49}, D_{45}, D_{40}, D_{25}, and D_1, respectively. The stock return is assumed to follow the LGNO with numerical setting: $S_{t_0} = 100$, $r = 0.1$, $\sigma = 0.3$, $T = 1$, $n = 50$.



(a) (b)

Figure 2: Convergence of Pricing Results: Log-Log Plots.
The log-log plots for the pricing results are shown in panel (a) with parameters $r = 0.1, S_{t_0} = 100, \sigma = 0.3, T = 1, n = 50, K = 0$. Panel (b) gives the log-log plot under the same settings except that $K = 120$. The x-axis and the y-axis denote $\ln h$ and the natural logarithm of the absolute pricing error, respectively. In panel (a) we use the closed-form pricing formula given by [12] to evaluate the exact value needed for calculating the pricing errors. The value 2.36828545183 obtained by the extrapolation method proposed by [21] is used as a proxy for the true option value in panel (b). In both panels, the circles, the triangles and the squares denote the results computed by the Benhamou algorithm, our 4th-order and 6th-order pricing algorithms, respectively. The dashed line, the gray line, and the solid black line denote the regression lines for the three algorithms.

		$\sigma = 0.2$		$\sigma = 0.3$	
K	r	Zhang	ours	Zhang	ours
90		12.596	12.595	13.952	13.952
100	0.05	5.763	5.762	7.946	7.944
110		1.990	1.989	4.072	4.070
90		13.831	13.831	14.984	14.983
100	0.09	6.777	6.776	8.829	8.827
110		2.546	2.545	4.697	4.695
90		15.642	15.642	16.513	16.512
100	0.15	8.409	8.408	10.210	10.208
110		3.556	3.555	5.730	5.729
		RMSE	0.001	RMSE	0.002

Table 2: Compare to Continuously Sampled Asian Option Prices.
This table shows that our pricing results converge to the prices of the continuously sampled Asian options as n large enough, where "Zhang" denotes the pricing results given by [36] with $S_0 = 100$, $T = 1$, and our pricing results are evaluated by the 4th-order converging algorithm with re-centering, where the numerical settings are $c = 1, n = 800$, and $h \approx 1.56 \times 10^{-4}$. The root-mean-square errors (RMSE) of our pricing results are given to show the superior of our algorithms.

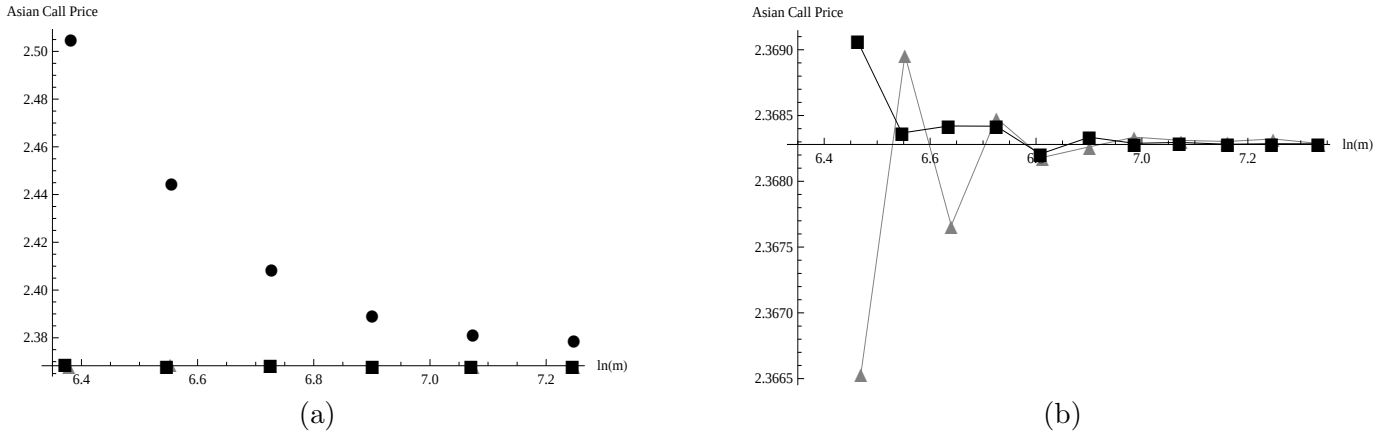


Figure 3: Convergence of Pricing Results for the Positive Strike Prices.
The numerical settings are the same as those in Figure 2 (b). The x- and the y-axes denote $\ln m$ and the pricing results, respectively. Since $h = 2c/m$, the meaning of the scale of x-axis, $\ln m = \ln 2c - \ln h$, is similar to $\ln h$. The circles, the triangles, and the squares denote the results computed by the Benhamou algorithm, our 4th-order and 6th-order algorithms, respectively.

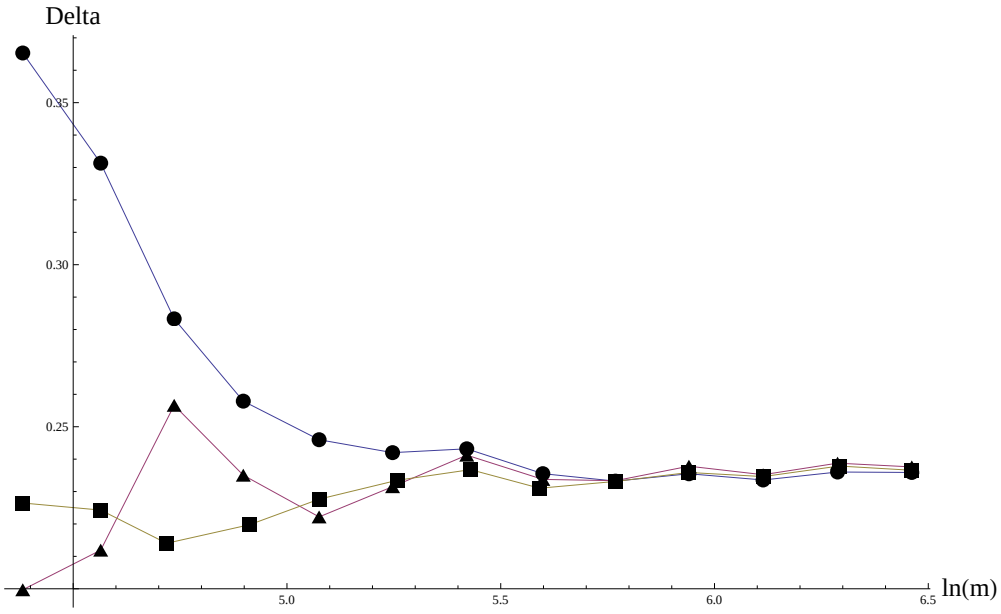


Figure 4: Convergence of Deltas.
The numerical settings are the same as those in Figure 2 (b). The x- and the y-axes denote $\ln m$ and the deltas, respectively. The circles, the triangles, and the squares denote the results computed by the Benhamou algorithm, our 4th-order and 6th-order algorithms, respectively.

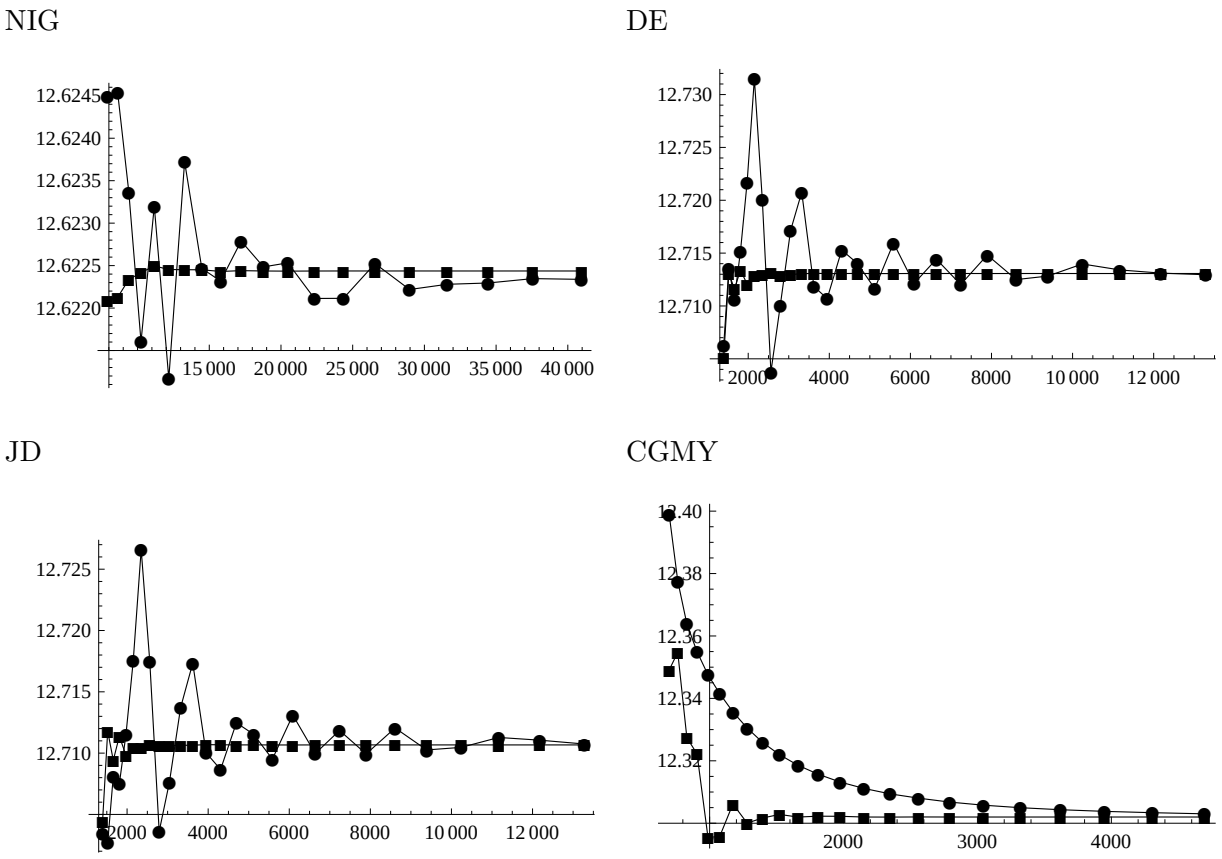


Figure 5: Convergence of Pricing Results under Lévy Processes.
Numerical settings are $r = 0.0367$, $S_{t_0} = 100$, $K = 90$, $T = 1$, $n = 12$, and $c = 8$, the same as those used in [19]. The parameters for NIG model are $\alpha = 6.1882$, $\beta = -3.8941$, $\delta = 0.1622$; for DE model, the parameters are $\sigma = 0.120381$, $p = 0.2071$, $\lambda = 0.330966$, $\eta_2 = 3.13868$, $\eta_1 = 9.65997$; for JD model, the parameters are $\sigma = 0.126349$, $\alpha = -0.390078$, $\lambda = 0.174814$, $\delta = 0.338796$; for CGMY model, the parameters are $C = 65.65$, $G = 47.38$, $M = 46.98$, $Y = -0.0719$. The x-axis denotes m and the y-axis denotes the pricing results. The circles and the squares denote the results computed by the Benhamou algorithm and our 4th-order algorithm, respectively.

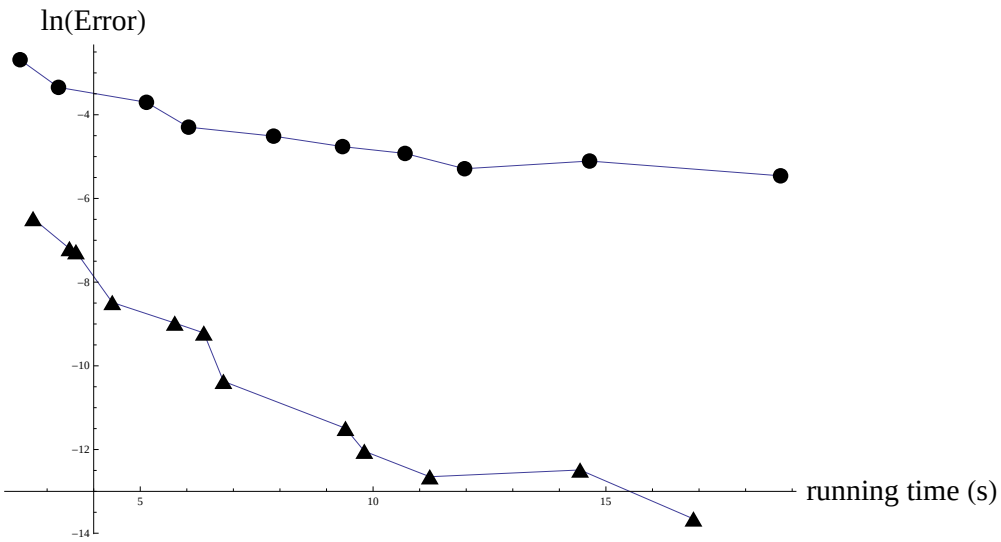


Figure 6: Running Time Comparison.
The numerical settings are the same as in Figure 2 (b). The x-axis and the y-axis denote the natural logarithm of absolute pricing error and the running time, respectively. The circles and the triangles denote the results computed by the Benhamou algorithm and our 4th-order pricing algorithm, respectively.